

Domande d'esame - risposte complete

Risposte in stile "esame scritto" alle 32 domande di esempio fornite dalla prof.ssa Iotti. Ordine originale conservato. Dove utile, formule e definizioni formali sono incluse esplicitamente.

Q1. Cosa si intende per "spazio metrico" e in che modo la "metrica euclidea su $\mathbb{R}$ " vi si relaziona?

Definizione di spazio metrico

Uno **spazio metrico** è una coppia (X, d) dove X è un insieme non vuoto e $d : X \times X \rightarrow \mathbb{R}$ è una funzione, detta **distanza** o **metrica**, che soddisfa i seguenti assiomi per ogni $x, y, z \in X$:

1. **Non-negatività:** $d(x, y) \geq 0$
2. **Identità degli indiscernibili:** $d(x, y) = 0 \Leftrightarrow x = y$
3. **Simmetria:** $d(x, y) = d(y, x)$
4. **Disuguaglianza triangolare:** $d(x, z) \leq d(x, y) + d(y, z)$

La metrica formalizza in modo astratto la nozione intuitiva di "distanza", indipendentemente dalla natura di X .

La metrica euclidea su $\mathbb{R}$

Su $X = \mathbb{R}$ la metrica naturale è $d(x, y) = |x - y|$. È immediato verificare che soddisfa i quattro assiomi (la triangolare segue dalla disuguaglianza $|a + b| \leq |a| + |b|$). $(\mathbb{R}, |\cdot|)$ è quindi uno spazio metrico, ed è il prototipo di tutti gli spazi metrici trattati nel corso.

Generalizzazione a $\mathbb{R}^N$

Su $\mathbb{R}^N$ la metrica euclidea è $d(x, y) = \|x - y\|_2 = \sqrt{\sum_i (x_i - y_i)^2}$, indotta dalla norma euclidea $\|\cdot\|_2$. Soddisfa gli assiomi (la triangolare segue dalla disuguaglianza di Cauchy-Schwarz).

Perché serve nel corso

Tutti i concetti di limite, continuità, convergenza, ottimizzazione richiedono una nozione di "vicinanza": la metrica è il minimo strumento che la rende formale. La discesa del gradiente, ad esempio, vive in $(\mathbb{R}^N, \|\cdot\|_2)$; le immagini digitali sono punti di $\mathbb{R}^N$ con $N = larghezza \times altezza \times canali$.

Q2. Come si definisce la "continuità" per una funzione @INL124@, e qual è il significato di "continuità separata"?

Continuità (definizione ε - δ)

$f : \mathbb{R}^N \rightarrow \mathbb{R}$ è **continua** in $x_0 \in \mathbb{R}^N$ se:

$$\forall \varepsilon > 0, \exists \delta > 0 : \forall x \in \mathbb{R}^N, \|x - x_0\| < \delta \Rightarrow |f(x) - f(x_0)| < \varepsilon$$

f è continua su $\mathbb{R}^N$ ($f \in C^0(\mathbb{R}^N)$) se è continua in ogni punto. In termini metrici: $d(x, x_0) < \delta \Rightarrow d(f(x), f(x_0)) < \varepsilon$. È la stessa idea della definizione di Cauchy nel caso $N = 1$, generalizzata da intervalli a "palle" in $\mathbb{R}^N$ (dischi se $N = 2$, sfere se $N = 3$, ecc.).

Continuità separata

Per $N = 2$, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ è **continua separatamente in x** se, fissato $y = y_0$, la funzione di una variabile $g(x) = f(x, y_0)$ è continua in ogni x . Analogamente continua separatamente in y . Si dice **separatamente continua** se lo è in entrambe le variabili.

Relazione tra le due nozioni

La continuità globale implica quella separata, ma non viceversa. Il controesempio classico è:

$$f(x, y) = xy/(x^2 + y^2) \text{ se } (x, y) \neq (0, 0), f(0, 0) = 0$$

Lungo gli assi f è identicamente 0 (continua separatamente in $(0, 0)$), ma lungo la retta $y = x$ vale $1/2$ per ogni $x \neq 0$, quindi non ha limite in $(0, 0)$ e non è globalmente continua.

Significato

La continuità globale richiede controllo del valore di f in *ogni* direzione di avvicinamento; quella separata solo lungo gli assi. È la prima manifestazione del fatto che il calcolo in più variabili non è la semplice giustapposizione di calcoli a una variabile.

Q3. Qual è la relazione tra "derivata direzionale", "derivata parziale" e "gradiente" nel calcolo in più variabili?

Derivata direzionale

Per $f: \mathbb{R}^N \rightarrow \mathbb{R}$, $x_0 \in \mathbb{R}^N$ e versore $v \in \mathbb{R}^N$ ($\|v\| = 1$), la **derivata direzionale** di f in x_0 lungo v è:

$$\partial f / \partial v(x_0) = \lim_{h \rightarrow 0} [f(x_0 + h \cdot v) - f(x_0)] / h$$

Misura il tasso istantaneo di variazione di f muovendosi da x_0 nella direzione v .

Derivata parziale

Caso particolare della direzionale: si sceglie $v = e_i$, l' i -esimo versore della base canonica. Si ottiene la **derivata parziale**:

$$\partial f / \partial x_i(x_0) = \lim_{h \rightarrow 0} [f(x_0 + h \cdot e_i) - f(x_0)] / h$$

cioè la derivata di f quando si lascia variare solo la coordinata i -esima e si tengono fisse le altre.

Gradiente

Il **gradiente** di f in x_0 è il vettore delle derivate parziali:

$$\nabla f(x_0) = (\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n)(x_0) \in \mathbb{R}^N$$

Relazione (per funzioni differenziabili)

Se f è differenziabile in x_0 , allora la derivata direzionale lungo qualunque v si esprime come prodotto scalare:

$$\partial f / \partial v(x_0) = \nabla f(x_0) \cdot v$$

Conseguenze fondamentali:

- Il gradiente "contiene" tutte le derivate direzionali.
- $|\partial f / \partial v| \leq \|\nabla f\|$ per Cauchy-Schwarz, con uguaglianza quando v è parallelo a ∇f . Quindi $\nabla f(x_0)$ **indica la direzione di massima crescita di f in x_0** , e $-\nabla f(x_0)$ quella di massima decrescita.
- Se $\nabla f(x_0) = 0$, x_0 è un punto critico (candidato a minimo, massimo o sella).

Ruolo nel corso

Il gradiente è il cuore dell'ottimizzazione: la discesa del gradiente sfrutta proprio la proprietà di massima decrescita per ridurre iterativamente la funzione di costo nelle reti neurali.

Q4. Descrivi il "metodo iterativo di discesa del gradiente" e la sua finalità.

Finalità

La **discesa del gradiente** (gradient descent, GD) è un metodo iterativo per trovare un minimo locale di una funzione differenziabile $f: \mathbb{R}^N \rightarrow \mathbb{R}$. È lo strumento standard per minimizzare funzioni di costo nell'apprendimento automatico, dove $\mathbf{f}$ rappresenta l'errore del modello sui dati di training in funzione dei parametri.

Idea

Da Q3 sappiamo che $-\nabla f(x)$ è la direzione di massima decrescita locale. Il metodo costruisce una successione x_k muovendosi a ogni passo nella direzione opposta al gradiente.

Algoritmo

1. Scegliere un punto iniziale $x_0 \in \mathbb{R}^N$ e un **learning rate** $\eta > 0$.
2. Per $k = 0, 1, 2, \dots$:

$$x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$$

1. Iterare finché un criterio di arresto è soddisfatto: $\|\nabla f(x_k)\| < tol$, numero massimo di iterazioni, o variazione di $\mathbf{f}$ sotto soglia.

Intuizione geometrica

Si immagini $\mathbf{f}$ come una superficie e una pallina che rotola: a ogni passo si sceglie la direzione di pendenza più ripida verso il basso e si avanza di un tratto proporzionale a η .

Ruolo del learning rate @@INL187@@

- η troppo piccolo $\rightarrow$ convergenza molto lenta.
- η troppo grande $\rightarrow$ oscillazioni o divergenza (si "salta" oltre il minimo).
- In pratica si usano η adattivi (Adam, RMSProp) o decay schedule.

Garanzie

Per $\mathbf{f}$ convessa e differenziabile con gradiente Lipschitziano, scegliendo η opportunamente piccolo, GD converge al minimo globale. In generale ($\mathbf{f}$ non convessa, come nelle reti neurali) converge solo a un punto critico, che può essere un minimo locale o un punto di sella.

Varianti rilevanti

- **Batch GD**: usa tutti i dati a ogni passo (costoso).
- **Stochastic GD (SGD)**: usa un singolo esempio (rumoroso ma veloce).
- **Mini-batch**: compromesso, standard nel deep learning.

Q5. Descrivi il metodo della discesa del gradiente. In quali casi potrebbe non convergere al minimo globale?

Metodo

(Vedi Q4 per la definizione formale e l'algoritmo.) Sintesi: aggiornamento $x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$ che sfrutta il fatto che $-\nabla f$ è la direzione di massima decrescita locale.

Casi di non convergenza al minimo globale

Pur essendo un metodo robusto, la discesa del gradiente garantisce solo la convergenza a un **punto critico**. Le situazioni problematiche sono: **1. Funzione non convessa con minimi locali multipli.** GD si ferma nel primo minimo locale incontrato, che può non essere globale. Esempio: $f(x) = x^4 - 3x^2 + x$. Le funzioni di costo delle reti neurali profonde sono fortemente non convesse. **2. Punti di sella.** In dimensione alta, i punti critici sono prevalentemente selle (Hessiana con autovalori di segno misto), non minimi. $\nabla f = 0$ in una sella ferma l'algoritmo anche se non è un minimo. Le tecniche moderne (momentum, Adam) aiutano a "scappare" dalle selle. **3. Plateau / regioni con gradiente quasi nullo.** Se $\|\nabla f\| \approx 0$ su una regione estesa, gli aggiornamenti diventano trascurabili e l'algoritmo "si blocca". Tipico nei modelli con sigmoidi saturate (vanishing gradient). **4. Learning rate inadeguato.**

- η troppo grande $\rightarrow$ oscillazioni attorno al minimo o divergenza ($f(x_{k+1}) > f(x_k)$).
- η troppo piccolo $\rightarrow$ convergenza lentissima, può non raggiungere il minimo nel budget computazionale disponibile.

5. Inizializzazione sfortunata. Punto di partenza in un bacino di attrazione "sbagliato": GD scende verso il minimo locale di quel bacino. La scelta di x_0 è cruciale; nelle reti neurali si usano inizializzazioni specifiche (Xavier, He) per evitarlo. **6. Funzione non differenziabile o gradiente non Lipschitz.** Se f non è differenziabile in qualche punto (es. ReLU in 0) si ricorre a sub-gradienti; se ∇f non è Lipschitz, le garanzie teoriche di convergenza saltano.

Conclusione

GD trova un minimo globale solo se la funzione è **convessa**. Per funzioni generiche garantisce solo un punto critico. Nel deep learning si accetta questa limitazione perché in pratica i minimi locali raggiunti producono modelli di buona qualità.

Q6. Come funziona un Single Layer Perceptron (SLP)? Quali sono i suoi limiti?

Architettura

Il **Single Layer Perceptron (SLP)** è il modello più semplice di rete neurale, introdotto da Rosenblatt nel 1958. Ha:

- N ingressi $x = (x_1, \dots, x_n)$
- N pesi $w = (w_1, \dots, w_n)$
- un termine di **bias** $b \in \mathbb{R}$
- una **funzione di attivazione** φ (storicamente la funzione gradino)
- una singola uscita y

Calcolo

L'uscita è:

$$z = w \cdot x + b = \sum_i w_i x_i + b$$
$$y = \varphi(z)$$

dove con la funzione gradino: $\varphi(z) = 1$ se $z \geq 0$, $\varphi(z) = 0$ altrimenti. Geometricamente, l'SLP definisce un **iperpiano** $w \cdot x + b = 0$ in $\mathbb{R}^N$ e classifica i punti in base al lato dell'iperpiano in cui si trovano. È un classificatore binario lineare.

Apprendimento (regola del perceptron)

Dato un dataset etichettato (x_i, t_i) , si aggiornano i pesi:

$$w \leftarrow w + \eta(t_i - y_i)x_i$$

$$b \leftarrow b + \eta(t_i - y_i)$$

Il **teorema di convergenza del perceptron** garantisce che, se i dati sono linearmente separabili, l'algoritmo converge in un numero finito di passi.

Limiti

1. Solo problemi linearmente separabili. L'SLP può risolvere AND, OR, NOT, ma **non lo XOR** (Minsky & Papert, 1969): non esiste un iperpiano che separi i punti $\{(0,0), (1,1)\}$ da $\{(0,1), (1,0)\}$. Questo limite è considerato la causa del primo "inverno dell'IA". **2. Una sola superficie di decisione lineare.** Non può approssimare funzioni non lineari né rappresentare classi non convesse o disconnesse. **3. Funzione di attivazione non differenziabile.** Il gradino non è differenziabile in 0 e ha gradiente nullo altrove, quindi non si può usare la backpropagation. Si usa la regola del perceptron, valida solo nel caso linearmente separabile.

Soluzione

Per superare questi limiti si introduce il **Multi-Layer Perceptron (MLP)**: aggiungendo strati nascosti e attivazioni non lineari (sigmoide, ReLU) si ottiene un approssimatore universale (vedi Q10).

Q7. Confronta le funzioni di attivazione ReLU, sigmoide e tanh in termini di proprietà matematiche e applicazioni.

Sigmoide

$$\sigma(z) = 1/(1 + e^{-z})$$
$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

- **Codominio:** $(0, 1)$.
- **Monotonia:** strettamente crescente.
- **Derivata:** $\sigma'(z) \in (0, 1/4]$, massima in $z = 0$.
- **Pro:** interpretabile come probabilità; storicamente la prima usata.
- **Contro:** **vanishing gradient** (per $|z|$ grande $\sigma' \approx 0$); output non centrato in 0 (rallenta la convergenza).
- **Uso:** output di classificatori binari; gate di LSTM/GRU.

Tangente iperbolica (tanh)

$$\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z}) = 2\sigma(2z) - 1$$
$$\tanh'(z) = 1 - \tanh^2(z)$$

- **Codominio:** $(-1, 1)$, **centrato in 0**.
- **Derivata:** $\in (0, 1]$, massima in $z = 0$.
- **Pro:** rispetto alla sigmoide, output centrato $\rightarrow$ gradienti più bilanciati, convergenza più rapida.
- **Contro:** soffre comunque di vanishing gradient per $|z|$ grande.
- **Uso:** hidden layer di RNN classiche, contesti in cui serve simmetria.

ReLU (Rectified Linear Unit)

$$ReLU(z) = \max(0, z)$$
$$ReLU'(z) = 1 \text{ se } z > 0, 0 \text{ se } z < 0 \text{ (non differenziabile in } 0)$$

- **Codominio:** $[0, +\infty)$.
- **Pro:**
 - costo computazionale minimo (un confronto);
 - **non satura per** $z > 0 \rightarrow$ niente vanishing gradient nella zona attiva;
 - induce **sparsità** (molti neuroni inattivi);
 - empiricamente accelera fortemente l'addestramento di reti profonde.
- **Contro:** **dying ReLU** - neuroni con $z \leq 0$ su tutti gli esempi hanno gradiente nullo permanentemente.
- **Uso:** hidden layer di tutte le moderne reti deep (CNN, transformer in alcune posizioni, MLP profondi). Varianti: Leaky ReLU, PReLU, ELU, GELU.

Tabella sintetica

Proprietà	Sigmoide	tanh	ReLU
Codominio	$(0,1)$	$(-1,1)$	$[0,+\infty)$
Centrata in 0	no	sì	no
Derivata massima	0.25	1	1
Vanishing gradient	sì	sì	no ($z > 0$)
Costo computazionale	alto (exp)	alto(exp)	basso
Differenziabile ovunque	sì	sì	no (in 0)

Conclusione

La scelta dipende dal ruolo del layer: ReLU è lo standard per gli strati nascosti in deep learning; sigmoide e softmax sono usate in output per classificazione; tanh sopravvive in alcune architetture ricorrenti.

Q8. Spiega il principio dell'algoritmo di backpropagation in un MLP con un layer nascosto.

Contesto

Un **Multi-Layer Perceptron (MLP)** con un layer nascosto è una funzione parametrica $f_\theta : \mathbb{R}^N \rightarrow \mathbb{R}^M$ definita da:

$$h = \varphi(W^{(1)}x + b^{(1)}) (\text{attivazione del layer nascosto})$$

$$\hat{y} = \psi(W^{(2)}h + b^{(2)}) (\text{uscita})$$

dove $\theta = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)})$ sono i parametri, φ e ψ sono funzioni di attivazione. Dato un dataset (x_i, t_i) , si vuole minimizzare una funzione di costo $L(\theta)$ (es. MSE, cross-entropy) tramite discesa del gradiente. Per applicarla servono $\partial L / \partial W^{(1)}, \partial L / \partial W^{(2)}, \partial L / \partial b^{(1)}, \partial L / \partial b^{(2)}$.

Principio: regola della catena

La **backpropagation** è l'applicazione sistematica della **chain rule** del calcolo differenziale per calcolare i gradienti di L rispetto a tutti i parametri in tempo lineare nel numero di pesi.

Forward pass

Dato un input x :

1. $z^{(1)} = W^{(1)}x + b^{(1)}, h = \varphi(z^{(1)})$
2. $z^{(2)} = W^{(2)}h + b^{(2)}, \hat{y} = \psi(z^{(2)})$
3. Calcolo della loss $L(\hat{y}, t)$.

Si memorizzano $x, z^{(1)}, h, z^{(2)}, \hat{y}$ (servono nel backward).

Backward pass

Si propagano gli "errori" (gradienti) all'indietro: **Layer di output:**

$$\delta^{(2)} = \partial L / \partial z^{(2)} = \partial L / \partial \hat{y} \cdot \psi'(z^{(2)})$$

$$\partial L / \partial W^{(2)} = \delta^{(2)} \cdot h^T$$

$$\partial L / \partial b^{(2)} = \delta^{(2)}$$

Layer nascosto:

$$\delta^{(1)} = (W^{(2)T} \delta^{(2)}) \odot \varphi'(z^{(1)}) (\odot = \text{prodotto element-wise})$$

$$\partial L / \partial W^{(1)} = \delta^{(1)} \cdot x^T$$

$$\partial L / \partial b^{(1)} = \delta^{(1)}$$

Aggiornamento

Con i gradienti calcolati, si aggiornano i parametri via gradient descent:

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \partial L / \partial W^{(l)}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \cdot \partial L / \partial b^{(l)}$$

Importanza

Prima della backprop (Rumelhart, Hinton, Williams 1986), addestrare reti multilayer era proibitivo. La backprop ha reso possibile il deep learning moderno: il costo di un passo di addestramento è circa $2\times$ il costo del forward pass, indipendentemente dalla profondità della rete.

Q9. Cos'è una funzione di costo e quale ruolo svolge nel processo di apprendimento? Fai un esempio con la MSE.

Definizione

Una **funzione di costo** (o **loss function**, **error function**) è una funzione $L : \Theta \rightarrow \mathbb{R}^+$ che misura quanto il modello parametrico f_θ si discosti dai dati di training. Formalmente, dato un dataset $D = (x_i, t_i)_{i=1}^n$:

$$L(\theta) = (1/n) \sum_i \ell(f_\theta(x_i), t_i)$$

dove $\ell(\hat{y}, t)$ è una **per-sample loss** (perdita su singolo esempio).

Ruolo nell'apprendimento

1. **Definisce l'obiettivo:** l'apprendimento si formalizza come $\theta^* = \operatorname{argmin}_\theta L(\theta)$.
2. **Guida l'ottimizzazione:** il gradiente $\nabla_\theta L$ indica la direzione di aggiornamento dei parametri (discesa del gradiente).
3. **Codifica il problema:** scegliere la loss equivale a definire cosa significa "errare" - regressione, classificazione e modelli generativi richiedono loss diverse.
4. **Determina le proprietà del modello:** convessità, robustezza agli outlier, calibrazione probabilistica dipendono dalla scelta di ℓ .

Esempio: Mean Squared Error (MSE)

Per problemi di **regressione** ($t_i \in \mathbb{R}$), la loss più comune è la **MSE** (o L2 loss):

$$L(\theta) = (1/n) \sum_i (f_\theta(x_i) - t_i)^2$$

Proprietà:

- Differenziabile ovunque, con gradiente $\partial \ell / \partial \hat{y} = 2(\hat{y} - t)$. Combinata con la chain rule, si presta perfettamente alla backpropagation.
- **Convessa nei parametri** se f_θ è lineare in θ (es. regressione lineare).
- **Penalizza fortemente gli errori grandi** (quadratico): un errore doppio pesa quattro volte. Ne consegue sensibilità agli outlier.
- **Interpretazione probabilistica:** minimizzare la MSE equivale alla **massima verosimiglianza** sotto l'ipotesi che $t = f(x) + \varepsilon$ con $\varepsilon \sim N(0, \sigma^2)$.

Confronto con L1 (MAE)

La **L1 loss** ($\ell = |\hat{y} - t|$) è più robusta agli outlier ma non differenziabile in 0 e ha gradiente costante (cattivo segnale di "vicinanza" al minimo). La scelta L1 vs L2 è un trade-off classico in ML.

Per classificazione

La MSE non è adatta a problemi di classificazione: la loss standard è la **cross-entropy**, derivata dalla massima verosimiglianza con distribuzione categoriale (vedi Q29).

Q10. Qual è il significato del teorema di approssimazione universale per le reti neurali?

Enunciato (Cybenko 1989, Hornik 1991)

Sia φ una funzione di attivazione **non polinomiale** continua (es. sigmoide, tanh, ReLU). Per ogni funzione continua $f : K \subset \mathbb{R}^N \rightarrow \mathbb{R}$ definita su un compatto K , e per ogni $\varepsilon > 0$, esistono un intero M , pesi $w_i \in \mathbb{R}^N$, bias $b_i \in \mathbb{R}$ e coefficienti $\alpha_i \in \mathbb{R}$ tali che la funzione

$$F(x) = \sum_{i=1}^M \alpha_i \cdot \varphi(w_i \cdot x + b_i)$$

(cioè un **MLP con un singolo layer nascosto** di larghezza M) approssimi f uniformemente:

$$\sup_{x \in K} |F(x) - f(x)| < \varepsilon$$

Significato concettuale

Le reti neurali con almeno un layer nascosto sono **approssimatori universali**: non c'è alcuna funzione continua "fuori portata" per un MLP, in linea di principio.

Cosa dice e cosa NON dice il teorema

Dice:

- L'esistenza di una rete che approssima f con precisione arbitraria.
- È sufficiente **un solo** layer nascosto.
- Vale per un'ampia classe di attivazioni (qualunque non polinomiale).

NON dice:

- **Quanti neuroni servono.** M può essere astronomicamente grande; per funzioni complesse cresce esponenzialmente con la dimensione (curse of dimensionality).
- **Come trovare i pesi.** Il teorema è esistenziale: garantisce che esistano parametri buoni, ma non come ottenerli (la backprop trova un minimo locale, non necessariamente quello che realizza l'approssimazione).
- **Generalizzazione:** il teorema riguarda l'approssimazione su K , non cosa succede su nuovi dati.
- **Profondità:** con un layer si può, ma reti **profonde** approssimano molte funzioni con esponenzialmente meno neuroni rispetto a reti shallow larghe (vantaggio espressivo della profondità).

Importanza storica e pratica

Il teorema fornisce la **giustificazione teorica** dell'uso delle reti neurali: cancella l'obiezione "le reti sono troppo limitate per essere universali" (fondata per l'SLP, vedi Q6, ma falsa per gli MLP). Tuttavia da solo non spiega il successo pratico del deep learning, che dipende anche da ottimizzazione, regolarizzazione e dati.

Q11. Quali sono i componenti fondamentali di un "perceptrone" o "neurone artificiale", e a cosa servono i "pesi" e il "bias"?

Componenti

Un **neurone artificiale** (modello di McCulloch–Pitts esteso da Rosenblatt) è la più piccola unità computazionale di una rete neurale. È composto da quattro elementi: **1. Ingressi** $x = (x_1, \dots, x_n)$. Vettore di valori reali in input. Possono essere feature dei dati grezzi (pixel di un'immagine, parole di un testo) o uscite di altri neuroni in strati precedenti. **2. Pesi** $w = (w_1, \dots, w_n)$. Un parametro reale per ciascun ingresso. Modulano l'importanza relativa di ogni input nella decisione del neurone. **3. Bias** $b \in \mathbb{R}$. Un parametro scalare aggiuntivo, indipendente dagli ingressi. **4. Funzione di attivazione** $\varphi : \mathbb{R} \rightarrow \mathbb{R}$. Funzione non lineare (sigmoide, tanh, ReLU, ecc.) applicata al risultato dell'aggregazione lineare.

Calcolo

Il neurone calcola:

$$z = w \cdot x + b = \sum_i w_i x_i + b(\text{pre-attivazione})$$
$$y = \varphi(z)(\text{attivazione})$$

Ruolo dei pesi

I pesi w_i rappresentano l'**importanza** di ciascun input:

- w_i grande in valore assoluto $\rightarrow$ l'input x_i ha forte influenza.
- $w_i \approx 0 \rightarrow$ l'input x_i viene ignorato.
- $w_i < 0 \rightarrow$ relazione inversa (più alto è x_i , meno il neurone è attivato).

Geometricamente, $\mathbf{w}$ è il **vettore normale** all'iperpiano di decisione $w \cdot x + b = 0$. L'apprendimento consiste nel trovare i pesi che producono la corretta separazione/regressione.

Ruolo del bias

Il bias b **trasla** l'iperpiano di decisione. Senza bias l'iperpiano passerebbe sempre per l'origine, limitando enormemente le funzioni rappresentabili. Esempio: un neurone deve attivarsi quando $x > 5$. Senza bias, con un solo peso w , la condizione $wx \geq 0$ non può catturare $x > 5$. Con $b = -5w$, la condizione $wx - 5w \geq 0 \Leftrightarrow x \geq 5$ diventa rappresentabile. In termini formali, il bias garantisce che la classe di funzioni rappresentabili sia **affine** anziché solo **lineare**, e questo è cruciale per la capacità espressiva.

Trucco implementativo

Spesso si "assorbe" il bias estendendo $x \leftarrow (1, x_1, \dots, x_n)$ e $w \leftarrow (b, w_1, \dots, w_n)$, riducendo il calcolo a un singolo prodotto scalare $w \cdot x$.

Q12. Definisci la "funzione di costo" e illustra la differenza tra "loss L1" (geometria del taxi) e "loss L2" (MSE).

Funzione di costo

(Definizione formale: vedi Q9.) È una funzione $L(\theta)$ che misura l'errore medio del modello sui dati di training; l'apprendimento la minimizza.

Loss L2 (Mean Squared Error)

Per $\hat{y}, t \in \mathbb{R}$:

$$\ell_{L2}(\hat{y}, t) = (\hat{y} - t)^2$$
$$L_{L2}(\theta) = (1/n) \sum_i (f_\theta(x_i) - t_i)^2$$

Per vettori $\hat{y}, t \in \mathbb{R}^m$, è il quadrato della **norma euclidea**: $\ell_{L2} = \|\hat{y} - t\|_2^2 = \sum_j (\hat{y}_j - t_j)^2$. Geometricamente corrisponde alla **distanza euclidea** al quadrato nel codominio.

Loss L1 (Mean Absolute Error, geometria del taxi)

$$\ell_{L1}(\hat{y}, t) = |\hat{y} - t|$$
$$L_{L1}(\theta) = (1/n) \sum_i |f_\theta(x_i) - t_i|$$

Per vettori $\ell_{L1} = \|\hat{y} - t\|_1 = \sum_j |\hat{y}_j - t_j|$. Corrisponde alla **distanza Manhattan** (geometria del taxi): la distanza che percorrerebbe un taxi muovendosi solo lungo le strade di una griglia rettangolare.

Differenze chiave

Proprietà	L1 (MAE)	L2 (MSE)
Geometria	taxi/Manhattan	euclidea
Crescita errore	lineare	quadratica
Differenziabile in 0	no (cuspidi)	sì
Gradiente	± 1 (costante)	$2(\hat{y} - t)$ (lineare)
Sensibilità outlier	bassa (robusto)	alta
Stimatore ottimo	mediana	media
Interpretazione MLE	rumore Laplace	rumore Gaussiano

Implicazioni pratiche

- **Outlier**: la L2 amplifica errori grandi (un errore di 10 conta 100), la L1 li tratta linearmente. In presenza di outlier, L1 dà modelli più robusti.
- **Ottimizzazione**: la L2 ha gradiente proporzionale all'errore, fornendo un segnale "graduale" che facilita la convergenza; la L1 ha gradiente di modulo costante, può causare oscillazioni vicino al minimo.
- **Compromesso**: la **Huber loss** combina L1 (per |errore| grande) e L2 (per |errore| piccolo) per avere robustezza + differenziabilità.

Quale scegliere

- L2: regressione standard, dati senza outlier, target gaussiani.
- L1: dati con outlier, mediana come quantità di interesse, problemi geometrici.

Q13. Elenca e descrivi brevemente almeno quattro "funzioni di attivazione" comunemente utilizzate nelle reti neurali artificiali.

Una **funzione di attivazione** $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ introduce non linearità nel neurone, trasformando la pre-attivazione $z = w \cdot x + b$ nell'output. Senza non linearità, una rete profonda collasserebbe in una semplice trasformazione lineare. Le quattro più comuni sono:

1. Sigmoidale (logistica)

$$\sigma(z) = 1/(1 + e^{-z}), \sigma(z) \in (0, 1)$$

- **Forma:** curva a "S" liscia, satura a 0 per $z \rightarrow -\infty$ e a 1 per $z \rightarrow +\infty$.
- **Derivata:** $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 1/4$.
- **Pro:** interpretabile come probabilità.
- **Contro:** vanishing gradient nelle code; output non centrato in 0.
- **Uso:** output di classificazione binaria, gate di LSTM/GRU.

2. Tangente iperbolica (tanh)

$$\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z}), \tanh(z) \in (-1, 1)$$

- **Forma:** come la sigmoide ma centrata e riscalata.
- **Derivata:** $\tanh'(z) = 1 - \tanh^2(z) \leq 1$.
- **Pro:** output centrato in 0 $\rightarrow$ gradienti più bilanciati.
- **Contro:** soffre comunque di vanishing gradient.
- **Uso:** hidden layer di RNN; storicamente preferita alla sigmoide negli hidden layer prima dell'avvento della ReLU.

3. ReLU (Rectified Linear Unit)

$$ReLU(z) = \max(0, z)$$

- **Forma:** lineare per $z > 0$, nulla per $z \leq 0$.
- **Derivata:** $ReLU'(z) = 1$ se $z > 0$, 0 se $z < 0$.
- **Pro:** nessuna saturazione per $z > 0$ (no vanishing gradient nella zona attiva); sparsità; calcolo banale.
- **Contro:** dying ReLU (neuroni sempre inattivi); non differenziabile in 0.
- **Uso:** standard de facto negli hidden layer di reti deep (CNN, MLP).

4. Softmax

$$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}, \sum_i \text{softmax}(z)_i = 1$$

- **Forma:** funzione vettoriale $\mathbb{R}^K \rightarrow \Delta^{K-1}$ (simplex). Generalizza la sigmoide a K classi.
- **Pro:** produce una distribuzione di probabilità sulle classi; combinata con cross-entropy ha gradienti semplici e ben condizionati.
- **Contro:** richiede normalizzazione su tutto il vettore; è una funzione vettoriale (non element-wise come le altre).
- **Uso:** layer di output per classificazione multiclasse.

Bonus: varianti moderne

- **Leaky ReLU:** $\max(\alpha z, z)$ con $\alpha \approx 0.01$, evita dying ReLU.
- **GELU:** $z \cdot \Phi(z)$ con Φ cdf gaussiana, smooth, usata nei transformer.
- **Swish/SiLU:** $z \cdot \sigma(z)$, simile a GELU.

Q14. Qual è il principio su cui si basa l'algoritmo di "backpropagation" e come mai è importante per l'addestramento di MLP?

Principio

La **backpropagation** è un algoritmo per calcolare in modo efficiente i gradienti della funzione di costo rispetto a tutti i parametri di una rete neurale. Si basa su due idee fondamentali: **1. Regola della catena (chain rule)**. Una rete neurale è una **composizione** di funzioni: $f_\theta = f_L \circ f_{L-1} \circ \dots \circ f_1$. Il gradiente di una composizione si calcola applicando ricorsivamente la chain rule:

$$\partial L / \partial \theta_k = \partial L / \partial f_L \cdot \partial f_L / \partial f_{L-1} \cdot \dots \cdot \partial f_{k+1} / \partial f_k \cdot \partial f_k / \partial \theta_k$$

2. Calcolo all'indietro (reverse-mode automatic differentiation). Aniché calcolare $\partial L / \partial \theta_k$ per ogni parametro separatamente (costoso), si propagano i gradienti **dall'output verso l'input** riutilizzando i calcoli intermedi. Il "segnale di errore" $\delta^{(l)} = \partial L / \partial z^{(l)}$ viene trasmesso dal layer L al layer 1 con la formula ricorsiva:

$$\delta^{(l)} = (W^{(l+1)T} \delta^{(l+1)}) \odot \varphi'(z^{(l)})$$

Da $\delta^{(l)}$ si ottengono direttamente $\partial L / \partial W^{(l)}$ e $\partial L / \partial b^{(l)}$.

Algoritmo (sintesi)

- 1. Forward pass:** calcola e memorizza $z^{(l)}, h^{(l)}$ per $l = 1, \dots, L$.
- 2. Loss:** calcola $L(\hat{y}, t)$ e $\delta^{(L)} = \partial L / \partial z^{(L)}$.
- 3. Backward pass:** per $l = L - 1, \dots, 1$ propaga $\delta^{(l)}$ indietro.
- 4. Aggiornamento:** $\theta \leftarrow \theta - \eta \cdot \nabla_\theta L$.

Importanza per gli MLP

1. Efficienza. Il costo è $O(P)$ con P numero totale di parametri, lo stesso ordine di un singolo forward pass. Senza backprop si dovrebbe stimare ogni $\partial L / \partial \theta_k$ con differenze finite (P forward pass), rendendo il training proibitivo per reti con milioni di parametri. **2. Generalità.** Funziona per qualunque architettura computabile come grafo di operazioni differenziabili (MLP, CNN, RNN, transformer). I framework moderni (PyTorch, TensorFlow) la implementano in modo automatico (autograd). **3. Abilitazione del deep learning.** Prima della backprop (Rumelhart, Hinton, Williams 1986) era noto solo come addestrare l'SLP, limitato ai problemi linearmente separabili. La backprop ha sbloccato l'addestramento degli MLP - quindi delle reti universali (vedi Q10) - e tutto ciò che ne è seguito. **4. Modularità.** Ogni layer espone solo un'interfaccia "forward + backward"; nuovi layer si possono inserire purché definiscano la loro derivata locale. Questo ha permesso un'esplosione di architetture sperimentali.

Limiti

- Richiede memoria proporzionale alla profondità (per memorizzare le attivazioni intermedie).
- Soffre di vanishing/exploding gradients in reti molto profonde (vedi Q31), problema mitigato da residual connections, normalizzazioni e attivazioni non saturanti.

Q15. Cosa afferma il "Teorema di approssimazione universale"?

Enunciato preciso (Cybenko 1989)

Sia $\varphi : \mathbb{R} \rightarrow \mathbb{R}$ una funzione continua, limitata, monotona crescente (detta "funzione sigmoideale" - la sigmoide standard ne è un esempio). Sia $K \subset \mathbb{R}^N$ un compatto. Allora le combinazioni lineari finite della forma

$$F(x) = \sum_{i=1}^M \alpha_i \cdot \varphi(w_i \cdot x + b_i)$$

con $M \in \mathbb{N}$, $\alpha_i, b_i \in \mathbb{R}$, $w_i \in \mathbb{R}^N$, sono **dense nello spazio** $C(K)$ delle funzioni continue su K . In altri termini, per ogni $f \in C(K)$ e ogni $\varepsilon > 0$ esistono M, α_i, w_i, b_i tali che

$$\sup_{x \in K} |F(x) - f(x)| < \varepsilon$$

L'estensione di Hornik (1991) generalizza il risultato a qualunque funzione di attivazione non polinomiale.

Interpretazione

Una **rete neurale feedforward con un solo layer nascosto** e numero sufficiente di neuroni può approssimare **qualunque funzione continua su un compatto** con precisione arbitraria. Le reti neurali sono dunque **approssimatori universali**.

Cosa garantisce

- **Esistenza** di una rete che approssima f entro ε .
- È sufficiente **un singolo layer nascosto**.
- Vale per un'ampia classe di attivazioni (qualunque non polinomiale: tanh, sigmoide, ReLU, ...).

Cosa NON garantisce

- **Numero di neuroni M**: nessun bound utile in pratica; può essere esponenziale nella dimensione N (curse of dimensionality).
- **Apprendibilità**: il teorema dice che esistono pesi buoni, non che la discesa del gradiente li trovi.
- **Generalizzazione**: l'approssimazione vale sui dati di training (su K), non garantisce buone performance su dati nuovi.
- **Profondità vs larghezza**: il teorema parla di reti shallow ($L = 1$). Risultati successivi mostrano che reti **deep** approssimano molte funzioni con esponenzialmente meno neuroni - vantaggio espressivo della profondità che il teorema universale non cattura.

Importanza

È la **giustificazione teorica** dell'uso delle reti neurali come modelli generali. Risponde all'obiezione storica (post Minsky-Papert) sulla limitatezza espressiva dei modelli neurali: i singoli percettroni sono limitati, ma un MLP con un layer nascosto è universale. Il successo pratico del deep learning, però, dipende da fattori (ottimizzazione, regolarizzazione, dati abbondanti, profondità) che il teorema da solo non spiega.

Q16. Descrivere le fasi principali del processo di "digitalizzazione delle immagini", specificando i concetti di "campionamento" e "quantizzazione".

Definizione

La **digitalizzazione** di un'immagine è il processo che converte una rappresentazione **continua** del mondo reale (luminosità che varia con continuità nello spazio e nei valori di intensità) in una **rappresentazione discreta** finita, manipolabile da un calcolatore. Si articola in tre fasi: acquisizione, campionamento, quantizzazione.

1. Acquisizione

Un sensore (CCD, CMOS) misura l'intensità luminosa proveniente dalla scena. Modello matematico: $f: \mathbb{R}^2 \rightarrow \mathbb{R}^+$, $(x, y) \mapsto f(x, y)$ con $f(x, y)$ proporzionale all'energia luminosa incidente sul punto (x, y) . Per immagini a colori si hanno 3 funzioni (R, G, B).

2. Campionamento (sampling)

È la **discretizzazione del dominio spaziale**. Si sceglie una griglia regolare di $M \times N$ punti e si valuta $\mathbf{f}$ in ciascun nodo:

$$f_d[i, j] = f(i \cdot \Delta x, j \cdot \Delta y), i = 0, \dots, M-1, j = 0, \dots, N-1$$

$\Delta x, \Delta y$ sono i passi di campionamento; $M \times N$ è la **risoluzione**. Ogni elemento della matrice risultante è un **pixel** (picture element). **Effetti del campionamento:**

- Risoluzione bassa $\rightarrow$ perdita di dettaglio, **aliasing** (artefatti come pattern moiré, bordi a scalini).
- **Teorema di Nyquist-Shannon:** per evitare aliasing, la frequenza di campionamento deve essere almeno doppia rispetto alla massima frequenza presente nel segnale. Si applica un filtro passa-basso (anti-aliasing) prima di campionare.

3. Quantizzazione

È la **discretizzazione del codominio** (dei valori di intensità). I valori continui di $\mathbf{f}$ vengono mappati in un insieme finito di L livelli:

$$f_q[i, j] = Q(f_d[i, j]) \in 0, 1, \dots, L-1$$

Tipicamente $L = 256$ (8 bit per pixel per canale). La quantizzazione introduce **errore di quantizzazione** (rumore) limitato a $\pm 0.5 \cdot \Delta L$. **Effetti della quantizzazione:**

- Pochi livelli (L piccolo) $\rightarrow$ **banding/posterizzazione**: bande visibili in transizioni morbide.
- Molti livelli $\rightarrow$ immagine fedele ma file più grande.

4. (Opzionale) Codifica

L'immagine quantizzata viene salvata in un formato (PNG, JPEG, ecc.) che può applicare compressione lossless o lossy.

Risultato

Un'immagine digitale è una matrice (o tensore se a colori) $I \in 0, 1, \dots, L-1^{M \times N \times C}$, dove $C = 1$ per scala di grigi e $C = 3$ per RGB.

Trade-off

La qualità dipende da campionamento (risoluzione spaziale) e quantizzazione (profondità di colore). Più sono fini, più l'immagine è fedele al continuo, ma più memoria richiede.

Q17. Cosa si intende per immagine digitale? Spiega i concetti di campionamento, quantizzazione e spazio colorimetrico.

Immagine digitale

Un'immagine digitale è una rappresentazione discreta e finita di una scena reale, ottenuta dal processo di digitalizzazione. Formalmente è una funzione

$$I : 0, \dots, M-1 \times 0, \dots, N-1 \rightarrow V^C$$

dove $M \times N$ è la risoluzione spaziale, C il numero di canali e $V = 0, 1, \dots, L-1$ l'insieme dei livelli quantizzati. Ogni elemento $I[i, j]$ è un **pixel**.

Campionamento

Discretizzazione del **dominio spaziale**: la luminosità continua viene misurata su una griglia regolare di punti (vedi Q16 per la definizione formale e il teorema di Nyquist). Esempio: una foto 1920×1080 ha $1920 \times 1080 \approx 2\text{milioni}$ di pixel. Aumentare la risoluzione $\rightarrow$ maggiore dettaglio spaziale, maggiore costo di memoria e calcolo.

Quantizzazione

Discretizzazione del **codominio dei valori**: i valori di luminanza continui vengono mappati in un insieme finito di livelli. Con 8 bit per canale si hanno $L = 256$ livelli ($0 = \text{nero}$, $255 = \text{biancomassimo}$ in scala di grigi). L'errore di quantizzazione è uniforme su $[-\Delta L/2, +\Delta L/2]$ con $\Delta L = 1/L$.

Spazio colorimetrico

Lo **spazio colorimetrico** (o color space) è il sistema matematico con cui si rappresenta il colore. Definisce come i valori numerici di un pixel vengono interpretati come colori percepiti. **Spazi principali:** **1. RGB (Red, Green, Blue)**. Modello additivo basato sui tre colori primari della sintesi additiva. Ogni pixel è una terna $(R, G, B) \in [0, 255]^3$. È lo standard per display (monitor, smartphone, TV) perché riflette il funzionamento dei pixel a sub-componenti R/G/B. **2. Scala di grigi (luminanza)**. Singolo canale $Y \in [0, 255]$. Conversione standard ITU-R BT.601: $Y = 0.299R + 0.587G + 0.114B$. I pesi tengono conto della sensibilità percettiva del verde > rosso > blu. **3. HSV (Hue, Saturation, Value)**. Modello percettivo: tinta (H), saturazione (S), luminosità (V). Utile per operazioni semantiche sul colore (es. "trova tutto ciò che è rosso"). **4. CMYK (Cyan, Magenta, Yellow, Key/Black)**. Sintesi sottrattiva. Standard per la stampa. **5. YCbCr / YUV**. Separa luminanza (Y) da cromaticanza (Cb, Cr). Sfruttato in compressione (JPEG, MPEG): l'occhio è meno sensibile alla cromaticanza, che può essere sotto-campionata. **6. CIE Lab**. Percettivamente uniforme: distanze nello spazio approssimano differenze di colore percepite. Utile per metriche di similarità.

Trade-off complessivo

La rappresentazione dell'immagine è il risultato di tre scelte:

- Quante celle sulla griglia? (campionamento - risoluzione spaziale)
- Quanti livelli per cella? (quantizzazione - profondità di colore)
- In che sistema? (spazio colorimetrico - interpretazione semantica)

Q18. Come si utilizza un istogramma per operazioni di elaborazione su immagini in scala di grigi?

Istogramma

L'**istogramma** di un'immagine in scala di grigi I con L livelli (es. $L = 256$) è il vettore $h : 0, \dots, L - 1 \rightarrow \mathbb{N}$ definito da:

$$h(k) = \#(i, j) : I[i, j] = k$$

cioè il **numero di pixel** che hanno intensità k . Spesso si usa la versione normalizzata $p(k) = h(k)/(M \cdot N)$, che è una distribuzione di probabilità (probabilità che un pixel scelto a caso abbia intensità k). L'istogramma fornisce una sintesi **statistica** della distribuzione dell'intensità nell'immagine, indipendente dalla disposizione spaziale dei pixel.

Cosa rivela

- **Esposizione:** istogramma concentrato a sinistra $\rightarrow$ immagine sotto-esposta (scura); concentrato a destra $\rightarrow$ sovraesposta.
- **Contrasto:** istogramma stretto e concentrato $\rightarrow$ basso contrasto; istogramma esteso su tutto il range $\rightarrow$ alto contrasto.
- **Picchi multipli:** presenza di regioni omogenee (es. soggetto vs sfondo).

Operazioni basate sull'istogramma

1. Stretching del contrasto (normalizzazione). Se l'istogramma occupa solo $[a, b] \subset [0, L - 1]$, si rimappa linearmente all'intero range:

$$I'[i, j] = (L - 1) \cdot (I[i, j] - a) / (b - a)$$

Espande l'istogramma a tutto il range, aumentando il contrasto. **2. Equalizzazione dell'istogramma.** Si applica come trasformazione la **funzione di distribuzione cumulativa** $CDF(k) = \sum_{j \leq k} p(j)$:

$$I'[i, j] = \text{round}((L - 1) \cdot CDF(I[i, j]))$$

L'istogramma risultante è approssimativamente **uniforme**: l'immagine usa in modo bilanciato tutti i livelli di grigio. Massimizza il contrasto globale ed è utile per immagini sotto-esposte/sovraesposte. **3. Sogliatura (thresholding).** Si sceglie una soglia T (manuale o automatica come **Otsu**, che massimizza la varianza inter-classe sull'istogramma) e si binarizza:

$$B[i, j] = 1 \text{ se } I[i, j] \geq T, \text{ else } 0$$

Operazione fondamentale per segmentazione semplice (separare oggetto da sfondo). L'istogramma bimodale facilita la scelta di T come "valle" tra i due picchi. **4. Histogram matching (specification).** Trasforma l'immagine in modo che il suo istogramma corrisponda a un istogramma target (es. quello di un'immagine di riferimento). Utile per uniformare colori tra immagini. **5. Operazioni puntuali (LUT).** Qualunque trasformazione $T : V \rightarrow V$ (es. correzione gamma, negativo, contrast stretching) può essere realizzata come **lookup table** indicizzata dall'istogramma; è un'operazione $O(1)$ per pixel.

Limiti

L'istogramma **ignora la struttura spaziale**: due immagini molto diverse possono avere lo stesso istogramma. Per operazioni che dipendono dal contesto locale servono filtri (es. Sobel) o istogrammi locali (CLAHE).

Q19. Spiega come funziona il filtro di Sobel e quale informazione estrae da un'immagine.

Obiettivo

Il **filtro di Sobel** è un operatore differenziale discreto usato per calcolare un'**approssimazione del gradiente** dell'intensità di un'immagine. Estrae informazione sui **bordi** (edge), cioè sui luoghi di forte variazione di luminosità.

Kernel di Sobel

Il filtro consiste in due kernel 3×3 , uno per ciascuna direzione assiale:

$$S_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

S_x approssima la derivata parziale $\partial I / \partial x$ (variazioni orizzontali); S_y approssima $\partial I / \partial y$ (variazioni verticali).

Operazione

Si applicano i due kernel via **convoluzione** (o cross-correlazione, vedi Q20) all'immagine I :

$$G_x = I * S_x(\text{gradiente orizzontale})$$

$$G_y = I * S_y(\text{gradiente verticale})$$

Dai due si ricavano:

$$M(i, j) = \sqrt{G_x^2(i, j) + G_y^2(i, j)} \text{ magnitudo del gradiente}$$
$$\theta(i, j) = \arctan2(G_y(i, j), G_x(i, j)) \text{ direzione del gradiente}$$

Informazione estratta

- $M(i, j)$: quanto rapidamente varia l'intensità nel pixel (i, j) . Valori alti = bordo; valori bassi = regione uniforme.
- $\theta(i, j)$: direzione perpendicolare al bordo (direzione di massima variazione dell'intensità).

Sogliando M si ottiene una mappa binaria dei bordi.

Perché funziona

Un bordo è una transizione rapida di intensità. Le derivate parziali $\partial I / \partial x$ e $\partial I / \partial y$ quantificano queste transizioni; la magnitudo del gradiente $|\nabla I| = \sqrt{(\partial I / \partial x)^2 + (\partial I / \partial y)^2}$ è invariante per rotazione e indica la "forza" del bordo.

Vantaggi del Sobel rispetto ad altre approssimazioni

I pesi (1, 2, 1) sull'asse ortogonale alla direzione di derivazione realizzano implicitamente un **smoothing gaussiano** 3×3 . Il Sobel combina quindi:

1. **Smoothing** (riduzione del rumore tramite media pesata).
2. **Derivazione** (rilevazione del bordo).

In un solo filtro, a parità di costo computazionale del Prewitt (che usa pesi uniformi (1,1,1) ed è più sensibile al rumore).

Connessione con il calcolo

Il Sobel può essere derivato dallo sviluppo di Taylor della derivata discreta. Il fatto che combini differenziazione e smoothing è una manifestazione concreta del fatto che, per estrarre derivate da segnali rumorosi, è essenziale filtrare prima.

Uso nel pipeline di Canny

Il Sobel è il secondo stadio del Canny edge detector (vedi progetto d'esame): dopo lo smoothing gaussiano, calcola gradiente e direzione che alimentano il non-maximum suppression.

Q20. Qual è la differenza tra convoluzione e cross-correlazione? Fornisci una definizione formale.

Cross-correlazione

La **cross-correlazione** tra due funzioni (o segnali, immagini) $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ è definita come:

$$(f \star g)(t) = \int f(\tau) \cdot g(t + \tau) d\tau (\text{continua})$$

$$(f \star g)[n] = \sum_k f[k] \cdot g[n + k] (\text{discreta})$$

In 2D (immagini), con I immagine e K kernel di dimensione $(2m + 1) \times (2n + 1)$:

$$(I \star K)[i, j] = \sum_{u=-m}^m \sum_{v=-n}^n I[i + u, j + v] \cdot K[u, v]$$

Convoluzione

La **convoluzione** è simile alla cross-correlazione ma con il **kernel "flippato"** (riflesso rispetto all'origine):

$$(f * g)(t) = \int f(\tau) \cdot g(t - \tau) d\tau (\text{continua})$$

$$(f * g)[n] = \sum_k f[k] \cdot g[n - k] (\text{discreta})$$

In 2D:

$$\begin{aligned} (I * K)[i, j] &= \sum_{u=-m}^m \sum_{v=-n}^n I[i - u, j - v] \cdot K[u, v] \\ &= \sum_{u, v} I[i + u, j + v] \cdot K[-u, -v] \end{aligned}$$

L'unica differenza con la cross-correlazione è il segno meno (o equivalentemente il flip del kernel su entrambi gli assi).

Differenza formale chiave

- **Cross-correlazione:** il kernel viene **applicato direttamente** - scivola sull'immagine e si fa il prodotto pesato.
- **Convoluzione:** il kernel viene **prima ribaltato** (flip orizzontale e verticale) e poi applicato.

Per kernel **simmetrici** (es. gaussiano, Laplaciano isotropo) le due operazioni coincidono. Per kernel **asimmetrici** (es. Sobel) i risultati differiscono per il segno o per uno specchio.

Proprietà che differenziano

Convoluzione:

- **Commutativa:** $f * g = g * f$.
- **Associativa:** $(f * g) * h = f * (g * h)$.
- **Distributiva** rispetto alla somma.
- **Teorema di convoluzione:** $Ff * g = Ff \cdot Fg$ (la trasformata di Fourier converte conv. in prodotto).

Cross-correlazione:

- **Non è commutativa:** $f \star g \neq g \star f$ in generale.
- **Non è associativa.**
- È molto usata per rilevare la posizione di un pattern in un segnale (template matching).

Nel deep learning

I "convolutional layer" delle CNN, nonostante il nome, implementano in realtà la **cross-correlazione**: si applica il kernel direttamente, senza flip. La distinzione è irrilevante operativamente perché i pesi del kernel sono **appresi**: se il flip fosse necessario, la rete imparerebbe il kernel già flippato.

Quando il flip conta

- **Image processing classico** (Canny, filtri, edge detection): la distinzione è significativa per la coerenza con la teoria del segnale. Implementare `convolve2d` invece di `correlate2d` cambia il segno di alcuni kernel (es. Sobel).
- **Sistemi LTI:** la risposta a un input è la convoluzione con la risposta impulsiva. La definizione di convoluzione è obbligatoria perché solo lei riproduce le proprietà fisiche dei sistemi.

Q21. Quali sono le proprietà principali dell'operazione di convoluzione?

Le proprietà della convoluzione $*$ la rendono lo strumento centrale per l'analisi dei sistemi lineari e per l'elaborazione di segnali e immagini.

1. Commutatività

$$f * g = g * f$$

L'ordine degli operandi non conta. Operativamente: applicare il kernel g all'immagine f produce lo stesso risultato di applicare f a g (utile per ottimizzazioni: si può scambiare per ridurre il costo).

2. Associatività

$$(f * g) * h = f * (g * h)$$

Permette di **comporre filtri**: applicare in sequenza i filtri g e h equivale ad applicare un singolo filtro $g * h$. Utile per pre-calcolare filtri composti e ridurre il numero di passaggi.

3. Distributività rispetto alla somma

$$f * (g + h) = (f * g) + (f * h)$$

La convoluzione è **lineare** in entrambi gli argomenti.

4. Linearità (omogeneità)

$$(\alpha f) * g = f * (\alpha g) = \alpha(f * g), \alpha \in \mathbb{R}$$

5. Esistenza dell'identità

La **delta di Dirac** δ è l'elemento neutro:

$$f * \delta = f$$

Nel discreto, $\delta[n] = 1$ se $n = 0$, 0 altrove. Convolvere per δ lascia il segnale invariato.

6. Differenziazione

$$(f * g)' = f' * g = f * g'$$

La derivata della convoluzione è la convoluzione con la derivata. Nel discreto/2D: applicare un filtro derivativo (Sobel, Laplaciano) è **lineare** e si può scambiare con altri filtri lineari. Conseguenza importante: si può calcolare $(I * G)' = I * G'$ (convolvere con la derivata del gaussiano), evitando di derivare numericamente l'immagine smussata.

7. Teorema di convoluzione (proprietà spettrale)

Sia F la trasformata di Fourier. Allora:

$$Ff * g = Ff \cdot Fg$$

$$Ff \cdot g = Ff * Fg$$

La convoluzione nel dominio spaziale corrisponde al **prodotto** nel dominio delle frequenze. Conseguenze:

- Filtri lineari sono caratterizzati dalla loro **risposta in frequenza**.
- Convoluzioni costose si possono calcolare via FFT in $O(N \log N)$ invece che $O(N^2)$.

8. Invarianza per traslazione

Se $T_a(f)(x) = f(x - a)$ è la traslazione, allora:

$$T_a(f * g) = T_a(f) * g = f * T_a(g)$$

La convoluzione **commuta** con le traslazioni. È la proprietà chiave che rende le CNN equivarianti per traslazione: lo stesso pattern in posizione diversa produce la stessa risposta traslata.

9. Separabilità (caso speciale 2D)

Se $K(x, y) = K_x(x) \cdot K_y(y)$, allora $I * K = (I * K_x) * K_y$ (prima filtro 1D orizzontale, poi 1D verticale). Riduce il costo da $O(M \cdot N \cdot k^2)$ a $O(M \cdot N \cdot k)$. Esempi separabili: gaussiano (la più importante), filtro media, Sobel separabile in $[1, 2, 1]^T$ per $[1, 0, -1]$.

10. Smoothing del massimo

Se $f, g \geq 0$ e g è una funzione di smussamento (es. gaussiana normalizzata), la convoluzione **non aumenta i massimi locali** e **riduce il rumore**.

Conseguenze pratiche

- **Modularità:** filtri si compongono con regole algebriche pulite.
- **Efficienza:** separabilità + FFT $\rightarrow$ calcolo veloce.
- **Teoria solida:** i filtri lineari sono completamente caratterizzati dal kernel, e la loro analisi è ben sviluppata in dominio sia spaziale sia frequenziale.

Q22. Illustra l'utilizzo dell'"istogramma dei livelli di grigio" in relazione agli "operatori puntuali" per la manipolazione di immagini luminanza.

Istogramma e operatori puntuali

Un **operatore puntuale** è una trasformazione $T: V \rightarrow V$ (con $V = 0, \dots, L-1$) applicata pixel per pixel: $I'[i, j] = T(I[i, j])$, **senza dipendenza dal vicinato**. Esempi: negativo, contrast stretching, correzione gamma, sogliatura. L'**istogramma** $h(k) = \#pixelconintensitàk$ (vedi Q18) è la distribuzione delle intensità. L'effetto di un operatore puntuale T è completamente prevedibile dall'istogramma: i pixel che condividono la stessa intensità subiscono la stessa trasformazione, quindi la nuova distribuzione si calcola da h e T senza guardare l'immagine.

Relazione formale

Se T è una funzione iniettiva, l'istogramma trasformato è:

$$h'(T(k)) = h(k)$$

Se T non è iniettiva (mappa più valori sullo stesso output):

$$h'(j) = \sum_{k:T(k)=j} h(k)$$

L'istogramma è quindi sia **strumento diagnostico** (rivela esposizione, contrasto) sia **dato di lavoro** per progettare operatori puntuali.

Operatori puntuali principali

1. Negativo (inversione).

$$T(k) = (L-1) - k$$

Usato in radiografia, per evidenziare strutture chiare su sfondo scuro. L'istogramma risulta speculare. **2. Correzione gamma.**

$$T(k) = (L-1) \cdot (k/(L-1))^\gamma$$

- $\gamma < 1$: scurisce le ombre, schiarisce i mezzi-toni $\rightarrow$ utile per immagini sovraesposte.
- $\gamma > 1$: schiarisce le ombre $\rightarrow$ utile per immagini scure.
- $\gamma = 1$: identità.

Compensa la non linearità percettiva della luminosità (sensibilità dell'occhio alle ombre). **3. Contrast stretching (normalizzazione).** Mappa $[a, b]$ (range effettivo dell'istogramma) su $[0, L-1]$:

$$T(k) = (L-1) \cdot (k-a)/(b-a)$$

Distribuisce l'istogramma su tutto il range, aumentando il contrasto. **4. Equalizzazione dell'istogramma.** Usa la CDF dell'istogramma come trasformazione:

$$T(k) = \text{round}((L-1) \cdot CDF(k))$$

Produce un istogramma approssimativamente uniforme. Massimizza il contrasto globale e l'entropia dell'immagine. **5. Sogliatura.**

$$T(k) = 0 \text{ se } k < T, L-1 \text{ se } k \geq T$$

Binarizza l'immagine. La scelta di T può essere manuale o automatica (metodo di Otsu, basato sulla varianza inter-classe sull'istogramma). **6. Compressione/dilatazione del range.** Mappature lineari a tratti per enfatizzare specifiche fasce di intensità.

Implementazione: lookup table (LUT)

Tutti gli operatori puntuali si implementano efficientemente come **LUT**: si pre-calcola un array $T[0..L-1]$ con i valori della trasformazione, e si applica $I'[i, j] = T[I[i, j]]$. Costo $O(M \cdot N)$ indipendente dalla complessità di T .

Vantaggi degli operatori puntuali

- **Costo computazionale minimo** ($O(1)$ per pixel via LUT).
- **Locali**: nessuna interazione tra pixel, parallelizzabili banalmente.
- **Reversibili** se T è iniettiva.

Limiti

- **Cieche al contesto spaziale**: non riducono il rumore (servono filtri), non rilevano bordi (servono operatori differenziali).
- L'istogramma non distingue oggetto da sfondo se condividono lo stesso range di intensità.

Conclusione

L'istogramma è la "fotografia statistica" dell'immagine; gli operatori puntuali sono le sue manipolazioni più semplici. La loro combinazione copre la maggior parte delle correzioni di luminosità e contrasto in fotografia, imaging medico e pre-processing.

Q23. In che modo i "polinomi di Taylor" vengono applicati per la creazione di "filtri di sharpening" come il filtro di Sobel o il filtro Laplaciano?

Polinomi di Taylor

Per una funzione differenziabile $f: \mathbb{R} \rightarrow \mathbb{R}$, lo **sviluppo di Taylor** in x_0 con incremento h è:

$$f(x_0 + h) = f(x_0) + h \cdot f'(x_0) + (h^2/2) \cdot f''(x_0) + (h^3/6) \cdot f'''(x_0) + O(h^4)$$

In più variabili (caso $f: \mathbb{R}^2 \rightarrow \mathbb{R}$):

$$\begin{aligned} f(x+h, y+k) &= f(x, y) + h \cdot \partial f / \partial x + k \cdot \partial f / \partial y \\ &+ (1/2)(h^2 \cdot \partial^2 f / \partial x^2 + 2hk \cdot \partial^2 f / \partial x \partial y + k^2 \cdot \partial^2 f / \partial y^2) + \dots \end{aligned}$$

Idea: derivate discrete da Taylor

Un'immagine digitale è una funzione discreta $I[i, j]$. Per estrarre informazione differenziale (bordi, dettagli, sharpening) servono **approssimazioni discrete** delle derivate. Si usano combinazioni lineari dei valori di pixel vicini, e i coefficienti si ricavano dallo sviluppo di Taylor.

Derivata prima (Sobel)

Per la derivata prima $\partial I / \partial x$ in (i, j) :

$$\begin{aligned} I[i, j+1] &= I[i, j] + (\partial I / \partial x) \cdot 1 + (1/2)(\partial^2 I / \partial x^2) + O(1) \\ I[i, j-1] &= I[i, j] - (\partial I / \partial x) \cdot 1 + (1/2)(\partial^2 I / \partial x^2) + O(1) \end{aligned}$$

Sottraendo:

$$\begin{aligned} I[i, j+1] - I[i, j-1] &= 2(\partial I / \partial x) + O(1) \\ \Rightarrow \partial I / \partial x &\approx (I[i, j+1] - I[i, j-1]) / 2 \end{aligned}$$

Questa è la **differenza centrata**, esatta a meno di $O(h^2)$. Il filtro corrispondente è $[-1, 0, +1]$. Per ridurre la sensibilità al rumore si **media** la stima centrata su tre righe verticali con pesi **(1, 2, 1)** (media pesata che dà peso doppio al centro):

$$S_x = \begin{vmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{vmatrix}$$

Questo è il **kernel di Sobel** orizzontale: derivata prima + smoothing gaussiano implicito, entrambi giustificati come approssimazioni di Taylor.

Derivata seconda (Laplaciano)

Sommando le due espansioni precedenti:

$$\begin{aligned} I[i, j+1] + I[i, j-1] &= 2 \cdot I[i, j] + (\partial^2 I / \partial x^2) + O(h^4) \\ \Rightarrow \partial^2 I / \partial x^2 &\approx I[i, j+1] - 2I[i, j] + I[i, j-1] \end{aligned}$$

Il **Laplaciano** è la somma delle derivate seconde: $\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$. Ne risulta il kernel:

$$L = \begin{vmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{vmatrix}$$

Variante con vicini diagonali (basata su un'espansione 2D più completa):

$$L8 = \begin{vmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{vmatrix}$$

Sharpening tramite Laplaciano

Il Laplaciano di un'immagine evidenzia i **punti di massima curvatura** dell'intensità (dove la derivata seconda è grande), che sono i bordi. Il filtro di **sharpening** combina l'immagine originale con il Laplaciano:

$$I_{sharp} = I - k \cdot \nabla^2 I \text{ con } k > 0$$

Il segno $-$ è dovuto al fatto che $\nabla^2 I$ è negativo nei picchi di intensità. L'effetto: i bordi vengono accentuati, l'immagine appare più "nitida". In forma di kernel singolo (per $k = 1$ con Laplaciano L_4):

$$H = \begin{vmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{vmatrix}$$

Sintesi del legame

- I valori di un pixel e dei suoi vicini stanno in relazione tramite Taylor.
- Combinazioni lineari di vicini, scelte secondo Taylor, producono **approssimazioni discrete delle derivate** (prime, seconde, miste).
- Filtri come **Sobel** (derivata prima $\rightarrow$ magnitudo del gradiente $\rightarrow$ bordi) e **Laplaciano** (derivata seconda $\rightarrow$ enhancement dei dettagli) sono applicazioni dirette di questa idea.
- Lo sharpening è la combinazione $I - k \cdot \nabla^2 I$, ottenuta sottraendo un termine basato sulla derivata seconda per accentuare le variazioni rapide.

In sintesi: **i filtri differenziali sono polinomi di Taylor discretizzati sui vicini di un pixel.**

Q24. Definisci l'operazione di convoluzione nel continuo e indicane le principali proprietà, sia in una che in due variabili.

Convoluzione in una variabile (continua)

Date due funzioni $f, g : \mathbb{R} \rightarrow \mathbb{R}$ integrabili, la loro **convoluzione** è:

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau) \cdot g(t - \tau) d\tau$$

Concettualmente: si "fa scorrere" g (riflessa attorno all'origine) sopra f , calcolando in ogni posizione t l'integrale del loro prodotto. Operativamente: per ogni t , è una **media pesata** di f con pesi g .

Convoluzione in due variabili (continua)

Date $f, g : \mathbb{R}^2 \rightarrow \mathbb{R}$:

$$(f * g)(x, y) = \int \int_{\mathbb{R}^2} f(u, v) \cdot g(x - u, y - v) du dv$$

Per immagini si usa la versione **discreta** ottenuta sostituendo gli integrali con somme (vedi Q20).

Proprietà principali

1. Commutatività.

$$f * g = g * f$$

Dimostrazione: cambio di variabile $\sigma = t - \tau$. **2. Associatività.**

$$(f * g) * h = f * (g * h)$$

Permette di comporre filtri lineari in qualunque ordine. **3. Distributività rispetto alla somma.**

$$f * (g + h) = (f * g) + (f * h)$$

4. Linearità (omogeneità).

$$(\alpha f) * g = \alpha(f * g), \forall \alpha \in \mathbb{R}$$

5. Identità: delta di Dirac. La convoluzione ha come elemento neutro la **distribuzione di Dirac** δ :

$$f * \delta = f$$

La δ "punta" nell'origine ed è il limite di funzioni concentrate (es. una gaussiana che si stringe). **6. Differenziazione.**

$$(f * g)' = f' * g = f * g'$$

La derivata della convoluzione è la convoluzione di una funzione con la derivata dell'altra. In 2D: $\partial(f * g)/\partial x = (\partial f/\partial x) * g = f * (\partial g/\partial x)$. **Conseguenza pratica:** per stimare $(I * G)'$ (derivata dell'immagine smussata con gaussiana) si convolve direttamente con G' (derivata della gaussiana), evitando la derivazione numerica dell'immagine. **7. Teorema di convoluzione.** La trasformata di Fourier F converte la convoluzione in prodotto:

$$Ff * g = Ff \cdot Fg$$

$$Ff \cdot g = (1/2\pi) Ff * Fg$$

Conseguenza: filtri lineari sono caratterizzati dalla loro **risposta in frequenza**; convoluzioni di grandi kernel si calcolano via FFT in $O(N \log N)$. **8. Invarianza per traslazione.** Se $T_a(f)(x) = f(x - a)$:

$$T_a(f * g) = T_a(f) * g = f * T_a(g)$$

Questa è la proprietà che giustifica matematicamente l'**equivarianza per traslazione** dei filtri convoluzionali in image processing e nelle CNN. **9. Supporto.** Se f ha supporto in A e g in B , allora $f * g$ ha supporto in $A + B = a + b : a \in A, b \in B$. La convoluzione "allarga" il supporto. **10. Caso particolare 2D - separabilità.** Se $g(x, y) = g_1(x) \cdot g_2(y)$, allora:

$$(f * g)(x, y) = ((f *_{x} g_1) *_{y} g_2)(x, y)$$

La convoluzione 2D si decompone in due convoluzioni 1D: enorme risparmio computazionale ($O(k^2) \rightarrow O(2k)$ per kernel $k \times k$). Esempio classico: la gaussiana 2D è il prodotto di due gaussiane 1D.

Conclusione

La convoluzione è un'operazione **lineare, invariante per traslazione** (LTI), che fornisce un linguaggio matematicamente solido per:

- elaborazione di segnali e immagini (filtri),
- analisi di sistemi fisici (risposta impulsiva),
- deep learning (CNN).

Le sue proprietà algebriche e spettrali permettono di costruire e analizzare filtri in modo modulare ed efficiente.

Q25. Quali sono le differenze strutturali e funzionali tra un MLP e una CNN?

MLP (Multi-Layer Perceptron)

Architettura feedforward general-purpose, composta da strati **fully-connected**: ogni neurone di un layer riceve in input **tutti** i neuroni del layer precedente. Calcolo per uno strato:

$$h = \varphi(W \cdot x + b) \quad W \in \mathbb{R}^{m \times n}, x \in \mathbb{R}^n, b \in \mathbb{R}^m$$

CNN (Convolutional Neural Network)

Architettura specializzata per dati con **struttura a griglia** (immagini, audio spettrale, video). I layer principali sono:

- **Convolutional layer**: applica kernel di piccole dimensioni (es. 3×3 , 5×5) a tutta l'immagine.
- **Pooling layer**: riduce la risoluzione spaziale.
- **Fully-connected layer** finali per classificazione.

Calcolo di un convolutional layer:

$$h[i, j, c'] = \varphi\left(\sum_{u, v, c} W[u, v, c, c'] \cdot x[i + u, j + v, c] + b[c']\right)$$

Differenze strutturali

Aspetto	MLP	CNN
Connettività	full (denso)	locale (kernel)
Condivisione pesi	no	sì (stesso kernel ovunque)
Numero parametri	molto alto	ridotto
Input nativo	vettore 1D	tensore 2D/3D ($H \times W \times C$)
Equivarianza traslazione	no	sì (per costruzione)
Gerarchia spaziale	no	sì (via pooling)

Differenze funzionali

1. Sfruttamento della struttura spaziale. L'MLP "appiattisce" l'immagine in un vettore, perdendo la nozione di vicinanza tra pixel. La CNN preserva la griglia 2D ed estrae feature locali (bordi, texture, parti di oggetto) che si combinano in modo gerarchico. **2. Numero di parametri.** MLP che processa un'immagine $224 \times 224 \times 3$:

- input flatten: 150528 neuroni
- primo hidden con 1000 unità: $\approx 1.5 \cdot 10^8$ parametri.

CNN con primo conv $3 \times 3 \times 3 \rightarrow 64$ filtri: 1728 parametri. La differenza in parametri è di **5 ordini di grandezza** o più.

3. Equivarianza per traslazione. Una CNN riconosce un pattern indipendentemente dalla sua posizione: lo stesso kernel viene applicato in ogni posizione. Un MLP deve "imparare" ogni traslazione separatamente. **4. Gerarchia di feature (con pooling/strides).** Layer iniziali $\rightarrow$ bordi e texture (feature locali, piccoli campi recettivi). Layer profondi $\rightarrow$ oggetti complessi (campi recettivi grandi). L'MLP non ha questa progressività spaziale. **5. Inductive bias.** La CNN incorpora ipotesi a priori adatte alle immagini: località, condivisione pesi, equivarianza. L'MLP è "neutro" - più flessibile in linea di principio ma molto meno efficiente sui dati con struttura.

Quando usare quale

- **MLP**: dati tabulari, vettoriali, senza struttura spaziale.
- **CNN**: immagini, audio (spettrogrammi), video (con estensione 3D), reti per dati 2D/3D in generale.
- **Trasformazione moderna**: i transformer competono con le CNN sulle immagini (Vision Transformer), eliminando la convoluzione esplicita ma reintroducendo equivalenze tramite positional encoding.

Conclusione

Una CNN è un MLP con vincoli architetturali (località, condivisione pesi) che incorporano l'**inductive bias** della struttura spaziale delle immagini. Questo le rende drasticamente più efficienti in parametri e dati necessari, oltre che equivariante per traslazione.

Q26. Perché le "Convolutional Neural Networks (CNN)" sono particolarmente efficaci per la gestione di "dati a griglia", come le immagini?

I dati a griglia (immagini, video, spettrogrammi audio, dati su grafi regolari) condividono tre proprietà che le CNN sfruttano per costruzione, e che invece un MLP deve "scoprire" da zero dai dati.

1. Località (locality)

Proprietà del dato: in un'immagine, le informazioni rilevanti per identificare un pattern sono **concentrate in una regione locale**. Un bordo, una texture, una parte di volto si manifestano in un intorno di pixel; pixel distanti tendono a essere indipendenti. **Sfruttamento CNN:** ogni kernel ha **dimensione piccola** (3×3 , 5×5) e analizza solo un intorno. Layer più profondi, applicandosi a feature già locali, vedono regioni più grandi (campo recettivo crescente). Questo costruisce una **gerarchia spaziale**: bordi $\rightarrow$ texture $\rightarrow$ parti $\rightarrow$ oggetti.

2. Condivisione di pesi (weight sharing)

Proprietà del dato: lo stesso pattern (es. un bordo verticale, un occhio) può apparire in qualunque posizione. La distribuzione di un pattern non dipende dalla sua posizione assoluta. **Sfruttamento CNN:** lo **stesso kernel viene applicato in ogni posizione** dell'immagine. Conseguenze:

- **Equivarianza per traslazione:** traslare l'input traslando l'output della stessa quantità.
- **Riduzione drastica dei parametri:** un kernel 3×3 con C canali e C' filtri ha $9 \cdot C \cdot C'$ pesi indipendentemente dalla risoluzione dell'immagine.
- **Generalizzazione spaziale:** se la rete impara a rilevare "occhio" in alto a sinistra, lo riconosce automaticamente anche in basso a destra.

3. Composizionalità gerarchica

Proprietà del dato: gli oggetti complessi sono **composti da parti** che a loro volta sono fatte di pattern più semplici. Un volto è fatto di occhi, naso, bocca; un occhio è fatto di iride, pupilla, palpebra; questi sono fatti di bordi e texture; questi sono fatti di variazioni di intensità. **Sfruttamento CNN:** l'architettura impila layer convoluzionali, con **pooling/strides** che riducono progressivamente la risoluzione spaziale.

- Layer 1: feature elementari (bordi, gradienti) su campi recettivi piccoli.
- Layer intermedi: combinazioni di feature (angoli, texture).
- Layer profondi: parti di oggetti, oggetti completi (campi recettivi che coprono tutta l'immagine).

Questa gerarchia rispecchia la struttura compositiva del dato.

4. Robustezza per costruzione

- **Invarianza per piccole traslazioni:** il pooling rende la rappresentazione poco sensibile a piccoli spostamenti.
- **Invarianza approssimata per piccole deformazioni:** convoluzione + pooling tollerano lievi distorsioni locali.

Queste invarianze sono **proprietà desiderate** per la classificazione di immagini.

Confronto quantitativo con un MLP

- Un MLP per immagini 1000×1000 richiederebbe pesi $O(10^6 \cdot n_{hidden})$, ingestibile.
- Una CNN con kernel 3×3 e poche decine di filtri ha pochi millesimi dei parametri, e generalizza meglio perché incorpora i bias corretti.

Estensioni

- **Video:** convoluzioni 3D (spazio + tempo).
- **Volumi medici (TC, RM):** convoluzioni 3D (volumetriche).
- **Audio:** convoluzioni 1D o 2D su spettrogrammi.
- **Grafi:** graph neural networks, generalizzazione delle CNN a domini non regolari.

Conclusione

L'efficacia delle CNN deriva dall'allineamento tra l'architettura e le **simmetrie naturali** del dato a griglia: località spaziale, equivarianza per traslazione, gerarchia compositiva. Imporre queste simmetrie come **inductive bias** anziché lasciarle apprendere riduce drasticamente i parametri necessari e migliora la generalizzazione.

Q27. Dare una breve descrizione dei minimi esempi di strati utilizzati in una CNN: "layer convoluzionale", "funzione di attivazione ReLU", "layer di pooling", e "dropout".

1. Layer convoluzionale (conv)

Operazione. Applica K filtri (kernel) di dimensione spaziale piccola (es. 3×3 o 5×5) e profondità pari ai canali in input. Per ciascun filtro $k = 1, \dots, K$:

$$y[i, j, k] = \sum_{u, v, c} W_k[u, v, c] \cdot x[i + u, j + v, c] + b_k$$

Iperparametri principali:

- **kernel size:** $f \times f$ (3×3 standard).
- **stride:** passo di scorrimento del kernel.
- **padding:** **valid** (no padding) o **same** (mantiene la risoluzione).
- **numero di filtri K :** definisce la profondità dell'output.

Output: un tensore $H' \times W' \times K$ con $H' = (H - f + 2p)/s + 1$, idem per W' . **Ruolo:** estrarre feature locali (bordi, texture, pattern complessi nei layer profondi). Pesi appresi dalla backpropagation.

2. Funzione di attivazione ReLU

Definizione: $ReLU(z) = \max(0, z)$. **Operazione.** Applicata element-wise dopo ogni conv layer:

$$a[i, j, k] = ReLU(y[i, j, k])$$

Ruolo:

- Introduce **non linearità**, condizione necessaria per l'espressività della rete (senza, una stack di conv è solo una conv lineare).
- Computazionalmente banale (un confronto).
- **Niente vanishing gradient** per $z > 0$ (gradiente costante = 1) $\rightarrow$ permette di addestrare reti profonde.
- Induce **sparsità** (molti neuroni inattivi $\rightarrow$ rappresentazioni più compatte).

Limite: dying ReLU (neuroni con $z \leq 0$ su tutti gli input restano permanentemente inattivi). Risolto da varianti (Leaky ReLU, PReLU, ELU).

3. Layer di pooling

Operazione. Riduce la risoluzione spaziale aggregando vicinati di pixel. Le due varianti più comuni: **Max pooling** 2×2 con stride 2:

$$y[i, j, k] = \max x[2i + u, 2j + v, k] : u, v \in \{0, 1\}$$

Average pooling 2×2 :

$$y[i, j, k] = (1/4) \sum_{u, v \in \{0, 1\}} x[2i + u, 2j + v, k]$$

Effetto:

- **Riduce la risoluzione spaziale** (tipicamente di $2 \times$).
- **Aumenta il campo recettivo effettivo** dei layer successivi.
- **Crea invarianza approssimata** per piccole traslazioni (la posizione esatta del pattern conta meno).
- **Riduce parametri e calcolo** nei layer successivi.

Iperparametri: dimensione finestra (2×2 standard), stride. Il **global average pooling** (media su tutta la mappa spaziale) è ormai preferito al pooling tradizionale prima della testa di classificazione, perché elimina i parametri dei layer fully-connected finali.

4. Dropout

Operazione. Durante l'addestramento, ogni neurone viene "spento" (impostato a 0) con probabilità p (es. $p = 0.5$):

$$y[i] = x[i] \cdot m[i] / (1 - p), m[i] \sim \text{Bernoulli}(1 - p)$$

La divisione per $1 - p$ mantiene l'**aspettativa** invariata (inverted dropout). In **inference**, il dropout è disattivato. **Ruolo:**

- **Regolarizzazione:** previene overfitting forzando la rete a non dipendere da specifici neuroni; impara rappresentazioni ridondanti.
- **Effetto ensemble:** equivale ad allenare implicitamente un'enorme famiglia di sotto-reti e a fare media in inference.

Posizionamento tipico: dopo i layer fully-connected; nelle moderne CNN (con BatchNorm e architetture residual) è meno usato nei conv, più comune nei classificatori finali e nei transformer.

Stack tipico di una CNN

Conv $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pooling $\rightarrow$... $\rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ Dropout $\rightarrow$ FC $\rightarrow$ Softmax

Layer aggiuntivi (utile per completezza)

- **Batch Normalization**: normalizza le attivazioni, accelera training e permette learning rate maggiori.
- **Skip/Residual connections** (ResNet): aggiungono x all'output, permettendo reti molto profonde (centinaia di layer).

Q28. Per quale tipo di dati sono concepite le "Recurrent Neural Networks (RNN)" e quale problematica affrontano?

Dati di destinazione: sequenze

Le **Recurrent Neural Networks (RNN)** sono architetture concepite per **dati sequenziali**, ovvero successioni $x = (x_1, x_2, \dots, x_T)$ in cui l'**ordine** degli elementi è informativo. Esempi:

- **Testo** (linguaggio naturale): sequenza di parole o caratteri.
- **Audio**: campioni o frame nel tempo.
- **Serie temporali**: dati finanziari, meteorologici, sensoriali.
- **Video**: sequenza di frame.
- **Sequenze biologiche**: DNA, proteine.

L'ipotesi chiave è che x_t dipenda dal **contesto precedente** $(x_1, \dots, x_{t-1})$.

Problema affrontato: dipendenze temporali

Le architetture feedforward (MLP, CNN) hanno due limiti su dati sequenziali:

1. **Input di lunghezza variabile**: una rete feedforward ha input fisso; sequenze di lunghezza diversa richiederebbero padding o reti separate.
2. **Mancanza di stato/memoria**: nessun meccanismo per "ricordare" informazioni di passi precedenti.

Le RNN risolvono entrambi introducendo uno **stato nascosto** che si aggiorna a ogni passo temporale.

Architettura base

A ogni passo temporale t :

$$h_t = \varphi(W_h \cdot h_{t-1} + W_x \cdot x_t + b)(\text{stato nascosto})$$
$$y_t = \psi(W_y \cdot h_t + b_y)(\text{output, opzionale})$$

dove:

- $h_t \in \mathbb{R}^d$ è lo **stato** al tempo t (riassunto del passato).
- W_h, W_x, W_y sono pesi **condivisi nel tempo** (come la condivisione spaziale nelle CNN, ma sull'asse temporale).
- φ è tipicamente **tanh**.

L'idea è che h_t codifichi tutto ciò che la rete "ricorda" di $(x_1, \dots, x_t)$. La condivisione dei pesi nel tempo dà alle RNN **equivarianza temporale** e le rende applicabili a sequenze di lunghezza arbitraria.

Apprendimento: backpropagation through time (BPTT)

La rete viene "srotolata" nel tempo (T copie del cell ricorrente collegate in catena), e la backpropagation classica si applica al grafo risultante. Il gradiente $\partial L / \partial W_h$ somma contributi da tutti i passi temporali.

Problematiche delle RNN classiche

1. **Vanishing/exploding gradients su lunghe sequenze**. Il gradiente attraversa T moltiplicazioni per la stessa matrice W_h :

- Se gli autovalori di W_h sono < 1 , il gradiente svanisce $\rightarrow$ la rete non impara dipendenze a lungo termine.
- Se > 1 , esplode $\rightarrow$ instabilità numerica.

Questo limita le RNN classiche a dipendenze di pochi passi. 2. **Difficoltà di addestramento**. La sequenzialità impedisce parallelizzazione efficace su GPU.

Soluzioni: LSTM e GRU

LSTM (Long Short-Term Memory, Hochreiter & Schmidhuber 1997) introduce una **cell state** c_t (memoria a lungo termine) e tre **gate** (forget, input, output) regolati da sigmoidi:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \text{ forgetgate}$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \text{ inputgate}$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \text{ outputgate}$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t)$$

I gate permettono di **mantenere o cancellare selettivamente** informazione nella cell state, mitigando il vanishing gradient e abilitando dipendenze a lungo termine (centinaia di passi). **GRU (Gated Recurrent Unit, Cho 2014)**: variante più semplice con due gate (reset, update), spesso prestazioni comparabili a LSTM con meno parametri.

Applicazioni storiche

- **Modelli di linguaggio** (predizione next-word).
- **Traduzione automatica** (encoder-decoder).
- **Speech recognition**.
- **Generazione di testo/musica**.

Stato dell'arte attuale

I **Transformer** (Vaswani 2017), basati sull'attenzione, hanno largamente soppiantato le RNN sulla maggior parte dei task NLP grazie a:

- parallelizzabilità,
- capacità di modellare dipendenze arbitrariamente lunghe,
- prestazioni superiori a parità di compute.

Le RNN/LSTM restano comunque rilevanti in contesti con vincoli di memoria o per streaming online.

Q29. Quali sono le principali caratteristiche della "funzione di costo cross-entropy" e della "funzione di attivazione softmax", e in quale contesto sono spesso utilizzate?

Contesto

La coppia **softmax** + **cross-entropy** è lo standard per la **classificazione multiclasse** (assegnare a un input una di K classi mutuamente esclusive). La softmax produce una distribuzione di probabilità sulle classi; la cross-entropy misura la distanza tra la distribuzione predetta e quella vera.

Funzione softmax

Data una pre-attivazione $z = (z_1, \dots, z_K) \in \mathbb{R}^K$ (i **logit**), la **softmax** restituisce un vettore di probabilità:

$$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j} \quad i = 1, \dots, K$$

Proprietà:

- $\text{softmax}(z)_i \in (0, 1)$ e $\sum_i \text{softmax}(z)_i = 1 \rightarrow$ distribuzione valida sul semplice Δ^{K-1} .
- **Generalizzazione della sigmoide**: per $K = 2$, equivale alla sigmoide applicata a $z_1 - z_2$.
- **Invariante per traslazione costante**: $\text{softmax}(z + c) = \text{softmax}(z)$ per ogni $c \in \mathbb{R}$. In pratica si sottrae $\max(z)$ per stabilità numerica (evitare overflow di e^z).
- **Differenziabile** ovunque, con derivata $\partial \text{softmax}(z)_i / \partial z_j = \text{softmax}(z)_i (\delta_{ij} - \text{softmax}(z)_j)$.

Interpretazione: la classe predetta $\arg\max_i \text{softmax}(z)_i$ è anche $\arg\max_i z_i$ (la softmax è monotona componente per componente nel logit massimo); il valore softmax esprime la **confidenza** della predizione.

Funzione di costo cross-entropy

Per una distribuzione vera $t = (t_1, \dots, t_K)$ e predetta $p = (p_1, \dots, p_K)$:

$$H(t, p) = - \sum_i t_i \cdot \log p_i$$

In classificazione multiclasse $\mathbf{t}$ è **one-hot** ($t_i = 1$ per la classe vera, 0 altrove), quindi:

$$H(t, p) = -\log p_{c_{\text{conc}}} = \text{classevera}$$

Si vuole $p_c \rightarrow 1$, equivalente a minimizzare $-\log p_c$. **Interpretazione probabilistica**. La cross-entropy è la **negative log-likelihood** sotto un modello categoriale; minimizzarla equivale alla **massima verosimiglianza** della classe vera. **Interpretazione informazionale**. Misura il numero medio di bit (o nat) necessari a codificare $\mathbf{t}$ usando il codice ottimale per $\mathbf{p}$. Quando $p = t$, è uguale all'entropia di $\mathbf{t}$; differenze positive misurano "l'inefficienza" di $\mathbf{p}$ come modello di $\mathbf{t}$. La differenza $H(t, p) - H(t, t) = D_{KL}(t||p)$ è la **divergenza di Kullback-Leibler**.

Combinazione softmax + cross-entropy

Si applica la softmax ai logit della rete, e si calcola la cross-entropy con l'etichetta one-hot:

$$L = -\log(e^{z_c} / \sum_j e^{z_j}) = -z_c + \log \sum_j e^{z_j}$$

Gradiente molto pulito (proprietà fondamentale):

$$\partial L / \partial z_i = \text{softmax}(z)_i - t_i = p_i - t_i$$

Il gradiente è la **differenza tra probabilità predetta e target**, identico nella forma a quello della MSE per regressione lineare. Questa struttura semplice rende la backpropagation molto efficiente e numericamente stabile.

Vantaggi della coppia softmax + cross-entropy

1. **Probabilità calibrate** (in linea di principio): la softmax produce probabilità interpretabili.
2. **Gradiente non saturante**: a differenza di MSE + softmax, dove i gradienti possono saturare, la coppia CE + softmax produce gradienti robusti.
3. **Stabilità numerica**: l'implementazione `log_softmax + nll_loss` evita di calcolare esplicitamente e^{z_i} problematici.
4. **Generalizzazione**: per $K = 2$ si riduce alla **binary cross-entropy** con sigmoide, che è il caso speciale standard.

Contesti di utilizzo

- **Classificazione multiclasse** (immagini ImageNet, documenti per argomento, fonemi).
- **Modelli di linguaggio** (predizione del prossimo token su un vocabolario di decine di migliaia di parole).
- **Politiche stocastiche in RL** (distribuzione su azioni).
- **Knowledge distillation** (cross-entropy tra distribuzioni di modelli).

Quando NON usare softmax + CE

- **Multi-label** (classi non esclusive): si usano sigmoide indipendenti + binary cross-entropy per ciascun output.
- **Regressione**: MSE/MAE sono appropriate, non CE.

Q30. Descrivi l'approccio dei modelli "Generative Adversarial Networks (GAN)".

Cosa sono

Le **Generative Adversarial Networks (GAN)**, introdotte da Goodfellow et al. nel 2014, sono un framework per **modelli generativi**: imparano a produrre nuovi campioni che sembrano provenire dalla distribuzione di un dataset di training (es. volti, paesaggi, animali fotorealistici). A differenza dei modelli generativi classici (Variational Autoencoder, modelli autoregressivi), le GAN **non massimizzano esplicitamente una verosimiglianza**: imparano per **contraddittorio**, mediante una "competizione" tra due reti.

Architettura

Due reti neurali in competizione: **1. Generatore** $G: \mathbb{R}^d \rightarrow \mathbb{R}^N$. Trasforma un vettore di rumore $z \sim p_z$ (es. gaussiano standard d -dim) in un campione $G(z)$ nello spazio dei dati (es. immagine). L'obiettivo del generatore è "ingannare" il discriminatore, producendo campioni indistinguibili da quelli reali. **2. Discriminatore** $D: \mathbb{R}^N \rightarrow [0, 1]$. Riceve un campione e produce la probabilità che sia **reale** (proveniente dal dataset) anziché **falso** (prodotto da G). È un classificatore binario.

Obiettivo (gioco minimax)

Le due reti sono addestrate alternativamente con obiettivi opposti:

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}} [\log D(x)] + E_{z \sim p_z} [\log(1 - D(G(z)))]$$

- **D** vuole massimizzare V : dare alta probabilità ai dati reali e bassa ai falsi.
- **G** vuole minimizzare V : produrre campioni che **D** ritiene reali.

L'equilibrio del gioco è il **punto di sella di Nash**: a convergenza, $p_G = p_{data}$ e $D(x) = 1/2$ ovunque (**D** non sa più distinguere).

Algoritmo di addestramento

A ogni iterazione si alterna: **Step 1 - aggiorna D** (per k step):

$$\nabla_{\theta_D} [(1/m) \sum \log D(x_i) + (1/m) \sum \log(1 - D(G(z_j)))]$$

Salita di gradiente. **Step 2 - aggiorna G**:

$$\nabla_{\theta_G} (1/m) \sum \log(1 - D(G(z_j)))$$

Discesa di gradiente. In pratica si usa la **non-saturating loss** per **G**:

$$\nabla_{\theta_G} - (1/m) \sum \log D(G(z_j))$$

che fornisce gradienti più forti quando **G** produce campioni facili da identificare come falsi.

Difficoltà

Le GAN sono notoriamente difficili da addestrare:

- **Instabilità**: il gioco minimax non ha garanzie di convergenza uniforme; oscillazioni e divergenza sono comuni.
- **Mode collapse**: **G** impara a produrre solo pochi campioni "facili" che ingannano **D**, ignorando la diversità del dataset.
- **Vanishing gradient per G**: se **D** è troppo bravo, $\log(1 - D(G(z)))$ satura e **G** non riceve segnale utile.
- **Bilanciamento**: **G** e **D** devono progredire in parallelo; se uno domina, l'apprendimento si blocca.

Varianti rilevanti

- **DCGAN** (Radford 2015): linee guida architetturali per usare conv layer in **G** e **D**, prima GAN visivamente convincente.
- **WGAN / WGAN-GP** (Arjovsky 2017): sostituiscono la cross-entropy con la **Wasserstein distance**, con gradient penalty per stabilità.
- **Conditional GAN (cGAN)**: condiziona **G** e **D** su un'etichetta (es. classe), permette generazione controllata.
- **CycleGAN**: traduzione tra domini senza coppie (es. cavalli $\leftrightarrow$ zebre).
- **StyleGAN** (Karras 2018): GAN ad alta risoluzione con controllo fine dello stile, alla base dei "deepfake" di volti.

Contesti di utilizzo

- **Generazione di immagini** fotorealistiche (volti, oggetti, scene).
- **Image-to-image translation** (sketch $\rightarrow$ foto, colorizzazione, super-resolution).
- **Domain adaptation e data augmentation**.
- **Generazione di musica, testo, video** (con varianti specifiche).
- **Modelli per simulazione fisica** (dati sintetici per training).

Stato dell'arte attuale

Negli ultimi anni le GAN sono state in larga parte soppiantate da **modelli a diffusione** (Stable Diffusion, DALL-E 2, Imagen) per la generazione di immagini, grazie a maggiore stabilità di training e qualità superiore. Le GAN restano competitive in domini specifici (volti ad alta risoluzione, traduzione tra stili) e in contesti dove la velocità di inferenza è prioritaria.

Conclusione

L'approccio GAN è elegante: invece di specificare esplicitamente la distribuzione p_{data} (difficile in alta dimensione), si delega a un discriminatore il compito di "valutare la qualità" dei campioni generati. Il framework adversarial ha ispirato decine di varianti e ha aperto la strada alla moderna AI generativa.

Q31. Quali problemi possono sorgere nella retropropagazione in MLP profondi (vanishing/exploding gradients)?

Origine del problema

Nella backpropagation in una rete profonda, il gradiente rispetto ai pesi del layer 1 si ottiene **moltiplicando** una catena di Jacobiani dei layer successivi:

$$\partial L / \partial W^{(l)} \propto J^{(L)} \cdot J^{(L-1)} \cdot \dots \cdot J^{(l+1)} \cdot \dots$$

dove $J^{(k)} = \partial h^{(k)} / \partial h^{(k-1)} = W^{(k)T} \cdot \text{diag}(\varphi'(z^{(k)}))$. Se la **norma** di questa catena cresce o decresce esponenzialmente con la profondità $L - l$, si manifestano due patologie speculari.

1. Vanishing gradient

Sintomo: $\|\partial L / \partial W^{(l)}\| \rightarrow 0$ per l lontano dall'output. I layer iniziali ricevono gradienti **trascurabili** e non vengono praticamente aggiornati. La rete impara solo gli ultimi layer, con capacità ridotta. **Cause:**

- **Attivazioni saturanti** (sigmoide, tanh): per $|z|$ grande, $\varphi'(z) \approx 0$. Il prodotto di molti φ' piccoli decade esponenzialmente.
- **Pesi inizializzati troppo piccoli:** gli autovalori delle Jacobiane hanno modulo < 1 , prodotto contrattivo.

Effetti pratici:

- Convergenza lentissima dei layer iniziali.
- Reti profonde (> 10 layer con sigmoidi) impossibili da addestrare.
- Storicamente la principale ragione per cui il deep learning ha avuto un "inverno" prima del 2006-2012.

2. Exploding gradient

Sintomo: $\|\partial L / \partial W^{(l)}\| \rightarrow \infty$. I gradienti diventano enormi, gli aggiornamenti $W \leftarrow W - \eta \cdot \nabla$ "esplodono", i pesi divergono fino a NaN. **Cause:**

- **Pesi inizializzati troppo grandi:** autovalori delle Jacobiane > 1 , prodotto espansivo.
- **Reti ricorrenti** su lunghe sequenze (BPTT moltiplica T Jacobiane della stessa cella).

Effetti pratici:

- Training instabile, loss che oscilla o esplode.
- Comune nelle RNN classiche su lunghe sequenze.

Caratterizzazione formale

Per una rete con pesi i.i.d. e attivazione lineare, il prodotto di L matrici random ha norma che cresce/decresce come λ^L dove λ è il raggio spettrale medio. La condizione $\lambda \approx 1$ (isometria) è fragile e richiede **inizializzazione attenta**.

Soluzioni / mitigazioni

1. Inizializzazione corretta dei pesi.

- **Xavier/Glorot** (per tanh/sigmoide): $W \sim N(0, 2/(n_{in} + n_{out}))$.
- **He** (per ReLU): $W \sim N(0, 2/n_{in})$.

Mantengono la varianza delle attivazioni e dei gradienti circa costante attraverso i layer. **2. Funzioni di attivazione non saturanti.**

- **ReLU** ha derivata costante 1 per $z > 0$, evitando il vanishing nella zona attiva. È la principale ragione del successo del deep learning moderno.
- Varianti (Leaky ReLU, ELU, GELU) migliorano oltre.

3. Normalizzazione delle attivazioni.

- **Batch Normalization** (Ioffe & Szegedy 2015): normalizza media e varianza delle pre-attivazioni in ogni mini-batch. Stabilizza le distribuzioni interne e mitiga vanishing/exploding.
- Varianti: Layer Norm (RNN, transformer), Instance Norm, Group Norm.

4. Skip / residual connections. Le **ResNet** (He et al. 2016) introducono connessioni $y = F(x) + x$. Il gradiente attraversa direttamente la skip, evitando di moltiplicare per Jacobiane piccole. Permette di addestrare reti con centinaia di layer. **5.**

Gradient clipping. Per exploding gradient, si **tronca** la norma del gradiente:

$$g \leftarrow g \cdot \min(1, c/\|g\|)$$

Standard per RNN/LSTM su lunghe sequenze. **6. Architetture con gating** (LSTM, GRU). I gate moltiplicativi creano percorsi additivi che evitano la moltiplicazione ripetuta di pesi. **7. Optimizer adattivi** (Adam, RMSProp). Normalizzano la magnitudo del gradiente per parametro, mitigando exploding e accelerando dove vanishing.

Conclusione

Vanishing e exploding gradients sono manifestazioni dello stesso problema strutturale: la moltiplicazione ripetuta di Jacobiane in reti profonde amplifica o estingue i gradienti esponenzialmente. Il deep learning moderno è in larga parte **l'insieme di tecniche** (ReLU, Xavier/He init, BatchNorm, ResNet, Adam, gradient clipping) che insieme stabilizzano questa moltiplicazione e permettono l'addestramento di reti molto profonde.

Q32. Spiega il funzionamento di una CNN, indicando il ruolo di ciascun tipo di layer (convoluzionale, pooling, lineare).

Funzionamento generale

Una **Convolutional Neural Network (CNN)** è una rete neurale specializzata per dati con struttura a griglia (immagini). Trasforma l'input - un tensore $H \times W \times C$ (altezza $\times$ larghezza $\times$ canali) - in un output (es. vettore di probabilità di classi) attraverso una sequenza di layer specializzati. L'idea di fondo è una **gerarchia di rappresentazioni**:

- Layer iniziali estraggono feature **locali ed elementari** (bordi, texture).
- Layer intermedi le combinano in **parti** (occhi, ruote, foglie).
- Layer finali aggregano le parti in **concetti globali** (volti, oggetti).

I tre tipi principali di layer hanno ruoli complementari.

1. Layer convoluzionale

Ruolo: estrazione di **feature locali**. **Operazione:** convolve l'input con K filtri di dimensione spaziale piccola (es. 3×3) e profondità pari ai canali in input. Per il filtro k -esimo:

$$y[i, j, k] = \sum_{u, v, c} W_k[u, v, c] \cdot x[i + u, j + v, c] + b_k$$

Perché funziona:

- **Località:** ogni pixel di output dipende solo da un piccolo intorno dell'input.
- **Condivisione pesi:** lo stesso kernel è applicato in tutte le posizioni $\rightarrow$ riduzione drastica dei parametri ed equivarianza per traslazione.
- **Filtri appresi:** i pesi sono ottimizzati dalla backpropagation per rilevare pattern utili per il task.

Tipicamente seguito da una funzione di attivazione (ReLU) e talvolta da batch normalization. **Iperparametri:** kernel size, numero di filtri, stride, padding.

2. Layer di pooling

Ruolo: down-sampling spaziale + invarianza locale. **Operazione:** riduce la risoluzione aggregando vicinati. Le forme più comuni sono **max pooling** e **average pooling** con finestra 2×2 , stride 2 (vedi Q27). **Perché funziona:**

- **Riduce la risoluzione** spaziale di un fattore 2 per dimensione $\rightarrow$ meno calcoli e parametri nei layer successivi.
- **Aumenta il campo recettivo effettivo:** i kernel dei layer successivi vedono regioni più grandi dell'immagine originale.
- **Crea invarianza approssimata** per piccole traslazioni: la posizione esatta di un pattern conta meno, conta che ci sia.
- **Costruisce la gerarchia spaziale:** alternando conv (estrazione) e pooling (compressione spaziale), la rete passa da feature locali ad alta risoluzione a feature globali a bassa risoluzione.

Nessun parametro apprendibile (oltre eventuale stride): il pooling è un'operazione fissa.

3. Layer lineare (fully-connected)

Ruolo: **classificazione finale** o regressione, combinando le feature globali estratte dai layer precedenti. **Operazione:** dopo l'ultimo blocco conv/pooling, le feature $H' \times W' \times C'$ vengono **appiattite** in un vettore. Uno o più layer fully-connected applicano:

$$y = \varphi(W \cdot x + b)$$

dove ogni neurone vede tutti i valori del vettore appiattito. L'ultimo layer FC produce i logit (es. uno per classe), seguiti da softmax per classificazione multiclasse. **Perché qui e non altrove:**

- I FC hanno **molti parametri** (ogni neurone è connesso a tutti gli input): metterli all'inizio sarebbe inefficiente.
- Alla fine della pipeline, le feature sono già state ridotte di dimensione e arricchite semanticamente: i FC integrano l'informazione globalmente per la decisione finale.

Tendenza moderna: sostituire (parzialmente) i FC finali con **global average pooling** (media spaziale per ogni canale $\rightarrow$ vettore di dimensione C'), riducendo drasticamente i parametri e l'overfitting (es. in ResNet).

Architettura tipica

```
Input (H x W x 3)
|
Conv -> ReLU -> Conv -> ReLU -> MaxPool      (blocco 1: feature locali)
|
Conv -> ReLU -> Conv -> ReLU -> MaxPool      (blocco 2: feature più astratte)
|
Conv -> ReLU -> Conv -> ReLU -> MaxPool      (blocco 3: feature semantiche)
|
GlobalAvgPool / Flatten
```

|
FC -> ReLU -> Dropout -> FC -> Softmax (testa di classificazione)

Layer ausiliari rilevanti

- **Activation function (ReLU)** dopo ogni conv/FC: introduce non linearità (vedi Q27).
- **Batch Normalization**: normalizza le attivazioni, accelera training e stabilizza.
- **Dropout**: regolarizzazione, soprattutto nei FC.
- **Skip / residual connections** (ResNet): permettono reti molto profonde.

Esempi storici di architetture

- **LeNet-5** (LeCun 1998): prima CNN di successo, riconoscimento di cifre manoscritte.
- **AlexNet** (2012): CNN profonda, vince ImageNet, dà avvio al deep learning moderno.
- **VGG** (2014): conv 3×3 impilati in profondità.
- **ResNet** (2015): residual connections, oltre 100 layer.
- **EfficientNet** (2019): scaling sistematico di profondità, larghezza, risoluzione.

Sintesi del ruolo dei layer

Layer	Ruolo	Parametri
Convolutionale	Estrazione feature locali, equivarianti	sì (kernel)
Pooling	Down-sampling, invarianza locale	no
Fully-connected	Aggregazione globale, classificazione finale	sì (matrice)
ReLU	Non linearità	no
BatchNorm	Normalizzazione attivazioni	sì (γ, β)
Dropout	Regolarizzazione	no

Conclusione

Una CNN funziona come un **estrattore gerarchico di feature**: i layer convoluzionali estraggono pattern locali sempre più astratti, i pooling riducono la risoluzione e accumulano contesto, e i layer fully-connected finali combinano tutto per la decisione. La sinergia tra questi layer, unita all'inductive bias della località/condivisione/composizionalità, spiega il successo delle CNN su dati a griglia. *Fine delle 32 domande.*